

앙상블(Ensemble)

머신러닝 앙상블이란 **여러개의 머신러닝 모델을 이용해 최적의 답을 찾아내는 기법**이다.

- 여러 모델을 이용하여 데이터를 학습하고, 모든 모델의 예측결과를 평균하여 예측

앙상블 기법의 종류

- 보팅 (Voting): 투표를 통해 결과 도출
- 배깅 (Bagging): 샘플 중복 생성을 통해 결과 도출
- 부스팅 (Boosting): 이전 오차를 보완하면서 가중치 부여
- 스택킹 (Stacking): 여러 모델을 기반으로 예측된 결과를 통해 meta 모델이 다시 한번 예측

참고자료 (블로그)

- [보팅\(Voting\)](#)
- [배깅\(Bagging\)](#)
- [부스팅\(Boosting\)](#)

실습을 위한 데이터셋 로드

```
In [5]: import pandas as pd
import numpy as np
from IPython.display import Image

np.set_printoptions(suppress=True)
```

```
In [6]: from sklearn.datasets import fetch_openml

data = fetch_openml(name="boston", version=1, as_frame=True)
```

데이터 로드

컬럼 소개

속성 수 : 13

- **CRIM**: 범죄율
- **ZN**: 25,000 평방 피트 당 주거용 토지의 비율
- **INDUS**: 비소매(non-retail) 비즈니스 면적 비율
- **CHAS**: 찰스 강 더미 변수 (통로가 하천을 향하면 1; 그렇지 않으면 0)
- **NOX**: 산화 질소 농도 (천만 분의 1)
- **RM**:주거 당 평균 객실 수
- **AGE**: 1940 년 이전에 건축된 자가 소유 점유 비율
- **DIS**: 5 개의 보스턴 고용 센터까지의 가중 거리
- **RAD**: 고속도로 접근성 지수
- **TAX**: 10,000 달러 당 전체 가치 재산 세율
- **PTRATIO** 도시 별 학생-교사 비율
- **B**: 1000 (Bk-0.63) ^ 2 여기서 Bk는 도시 별 검정 비율입니다.
- **LSTAT**: 인구의 낮은 지위
- **MEDV**: 자가 주택의 중앙값 (1,000 달러 단위)

```
In [9]: df = pd.DataFrame(data['data'], columns=data['feature_names'])
```

```
In [10]: df['MEDV'] = data['target']
```

```
In [11]: df.head()
```

```
Out[11]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

train / test 데이터를 분할 합니다.

```
In [13]: from sklearn.model_selection import train_test_split
```

```
In [14]: x_train, x_test, y_train, y_test = train_test_split(df.drop('MEDV', axis=1), df['MEDV'], random_state=42)
```

```
In [15]: x_train.shape, x_test.shape
```

```
Out[15]: ((379, 13), (127, 13))
```

```
In [16]: x_train.head()
```

```
Out[16]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
182	0.09103	0.0	2.46	0	0.4880	7.155	92.2	2.7006	3	193.0	17.8	394.12	4.82
155	3.53501	0.0	19.58	1	0.8710	6.152	82.6	1.7455	5	403.0	14.7	88.01	15.02
280	0.03578	20.0	3.33	0	0.4429	7.820	64.5	4.6947	5	216.0	14.9	387.31	3.76
126	0.38735	0.0	25.65	0	0.5810	5.613	95.6	1.7572	2	188.0	19.1	359.29	27.26
329	0.06724	0.0	3.24	0	0.4600	6.333	17.2	5.2146	4	430.0	16.9	375.21	7.34

```
In [17]: y_train.head()
```

```
Out[17]: 182    37.9
          155    15.6
          280    45.4
          126    15.7
          329    22.6
          Name: MEDV, dtype: float64
```

평가 지표 만들기

MSE(Mean Squared Error)

$$\left(\frac{1}{n}\right) \sum_{i=1}^n (y_i - x_i)^2$$

예측값과 실제값의 차이에 대한 **제곱**에 대하여 평균을 낸 값

MAE (Mean Absolute Error)

$$\left(\frac{1}{n}\right) \sum_{i=1}^n |y_i - x_i|$$

예측값과 실제값의 차이에 대한 **절대값**에 대하여 평균을 낸 값

RMSE (Root Mean Squared Error)

$$\sqrt{\left(\frac{1}{n}\right) \sum_{i=1}^n (y_i - x_i)^2}$$

예측값과 실제값의 차이에 대한 **제곱**에 대하여 평균을 낸 뒤 **루트**를 씌운 값

sklearn의 평가지표 활용하기

```
In [29]: from sklearn.metrics import mean_absolute_error, mean_squared_error
```

모델별 성능 확인을 위한 함수

```
In [31]: import matplotlib.pyplot as plt
import seaborn as sns

my_predictions = {}

colors = ['r', 'c', 'm', 'y', 'k', 'khaki', 'teal', 'orchid', 'sandybrown',
          'greenyellow', 'dodgerblue', 'deepskyblue', 'rosybrown', 'firebrick',
          'deeppink', 'crimson', 'salmon', 'darkred', 'olivedrab', 'olive',
          'forestgreen', 'royalblue', 'indigo', 'navy', 'mediumpurple', 'chocolate',
          'gold', 'darkorange', 'seagreen', 'turquoise', 'steelblue', 'slategray',
          'peru', 'midnightblue', 'slateblue', 'dimgray', 'cadetblue', 'tomato'
          ]

def plot_predictions(name_, pred, actual):
    df = pd.DataFrame({'prediction': pred, 'actual': y_test})
    df = df.sort_values(by='actual').reset_index(drop=True)

    plt.figure(figsize=(12, 9))
    plt.scatter(df.index, df['prediction'], marker='x', color='r')
    plt.scatter(df.index, df['actual'], alpha=0.7, marker='o', color='black')
    plt.title(name_, fontsize=15)
    plt.legend(['prediction', 'actual'], fontsize=12)
    plt.show()

def mse_eval(name_, pred, actual):
    global predictions
    global colors

    plot_predictions(name_, pred, actual)

    mse = mean_squared_error(pred, actual)
    my_predictions[name_] = mse

    y_value = sorted(my_predictions.items(), key=lambda x: x[1], reverse=True)

    df = pd.DataFrame(y_value, columns=['model', 'mse'])
    print(df)
```

```

min_ = df['mse'].min() - 10
max_ = df['mse'].max() + 10

length = len(df)

plt.figure(figsize=(10, length))
ax = plt.subplot()
ax.set_yticks(np.arange(len(df)))
ax.set_yticklabels(df['model'], fontsize=15)
bars = ax.barh(np.arange(len(df)), df['mse'])

for i, v in enumerate(df['mse']):
    idx = np.random.choice(len(colors))
    bars[i].set_color(colors[idx])
    ax.text(v + 2, i, str(round(v, 3)), color='k', fontsize=15, fontweight='bold')

plt.title('MSE Error', fontsize=18)
plt.xlim(min_, max_)

plt.show()

def remove_model(name_):
    global my_predictions
    try:
        del my_predictions[name_]
    except KeyError:
        return False
    return True

def plot_coef(columns, coef):
    coef_df = pd.DataFrame(list(zip(columns, coef)))
    coef_df.columns = ['feature', 'coef']
    coef_df = coef_df.sort_values('coef', ascending=False).reset_index(drop=True)

    fig, ax = plt.subplots(figsize=(9, 7))
    ax.barh(np.arange(len(coef_df)), coef_df['coef'])
    idx = np.arange(len(coef_df))
    ax.set_yticks(idx)
    ax.set_yticklabels(coef_df['feature'])

```

```
fig.tight_layout()  
plt.show()
```

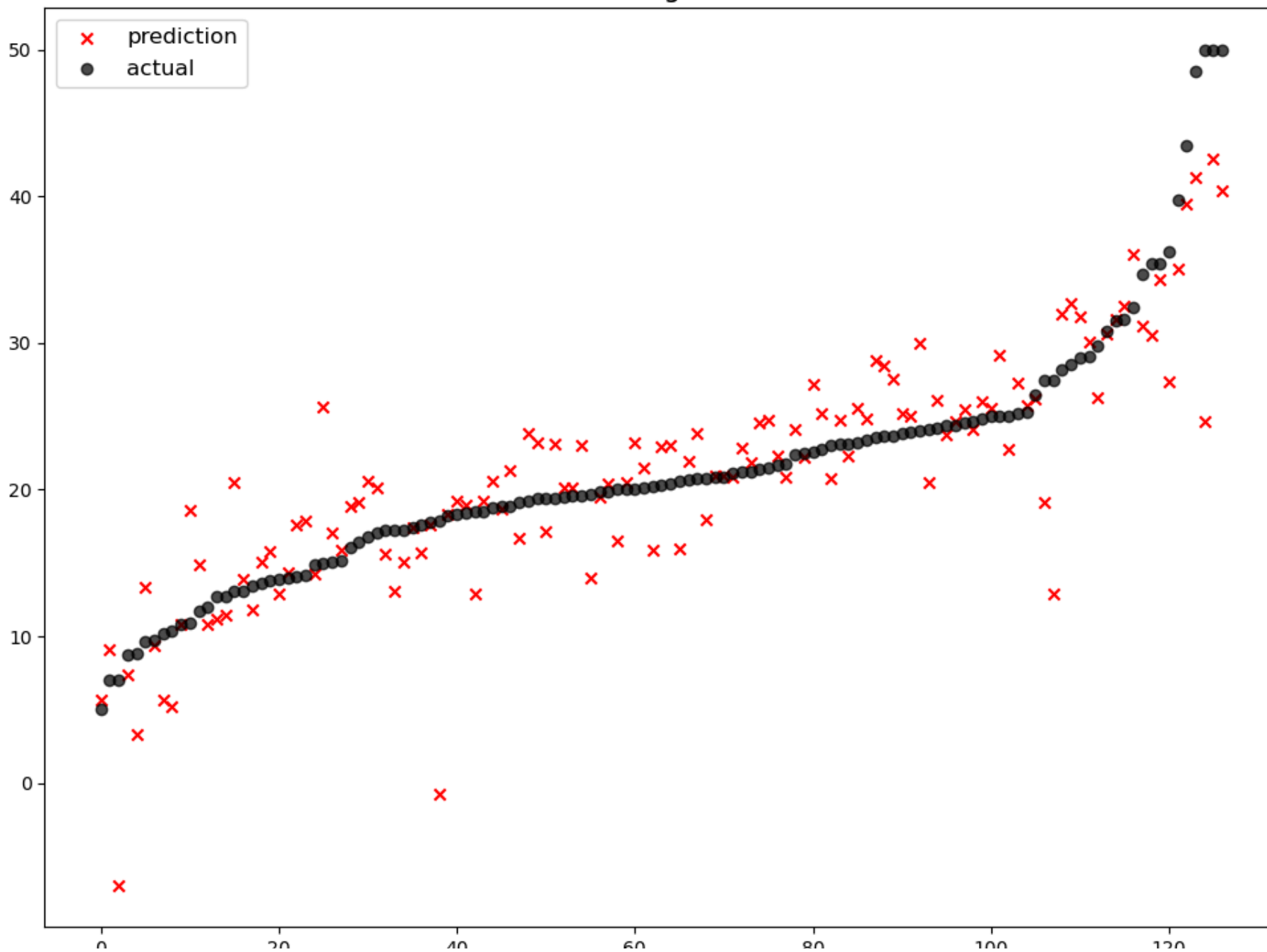
단일 회귀예측 모델 (지난 시간)

```
In [33]: from sklearn.linear_model import LinearRegression  
        from sklearn.linear_model import Ridge  
        from sklearn.linear_model import Lasso  
        from sklearn.linear_model import ElasticNet  
        from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler  
        from sklearn.pipeline import make_pipeline  
        from sklearn.preprocessing import PolynomialFeatures
```

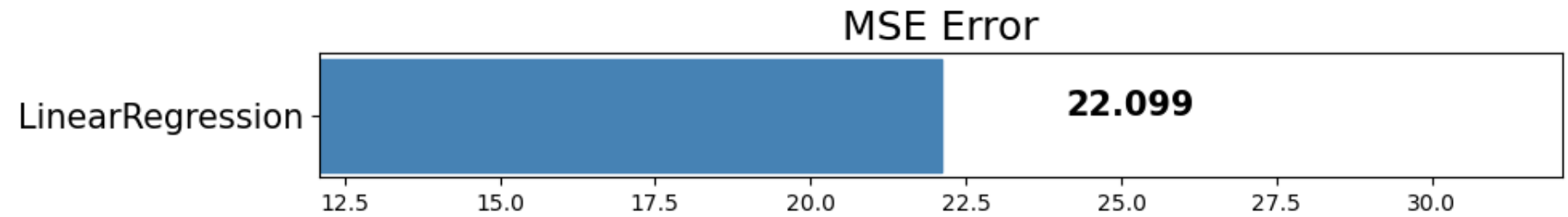
```
In [34]: # astype으로 타입 변경  
        x_train = x_train.astype(float)  
        x_test = x_test.astype(float)
```

```
In [35]: linear_reg = LinearRegression(n_jobs=-1)  
        linear_reg.fit(x_train, y_train)  
        pred = linear_reg.predict(x_test)  
        mse_eval('LinearRegression', pred, y_test)
```

LinearRegression

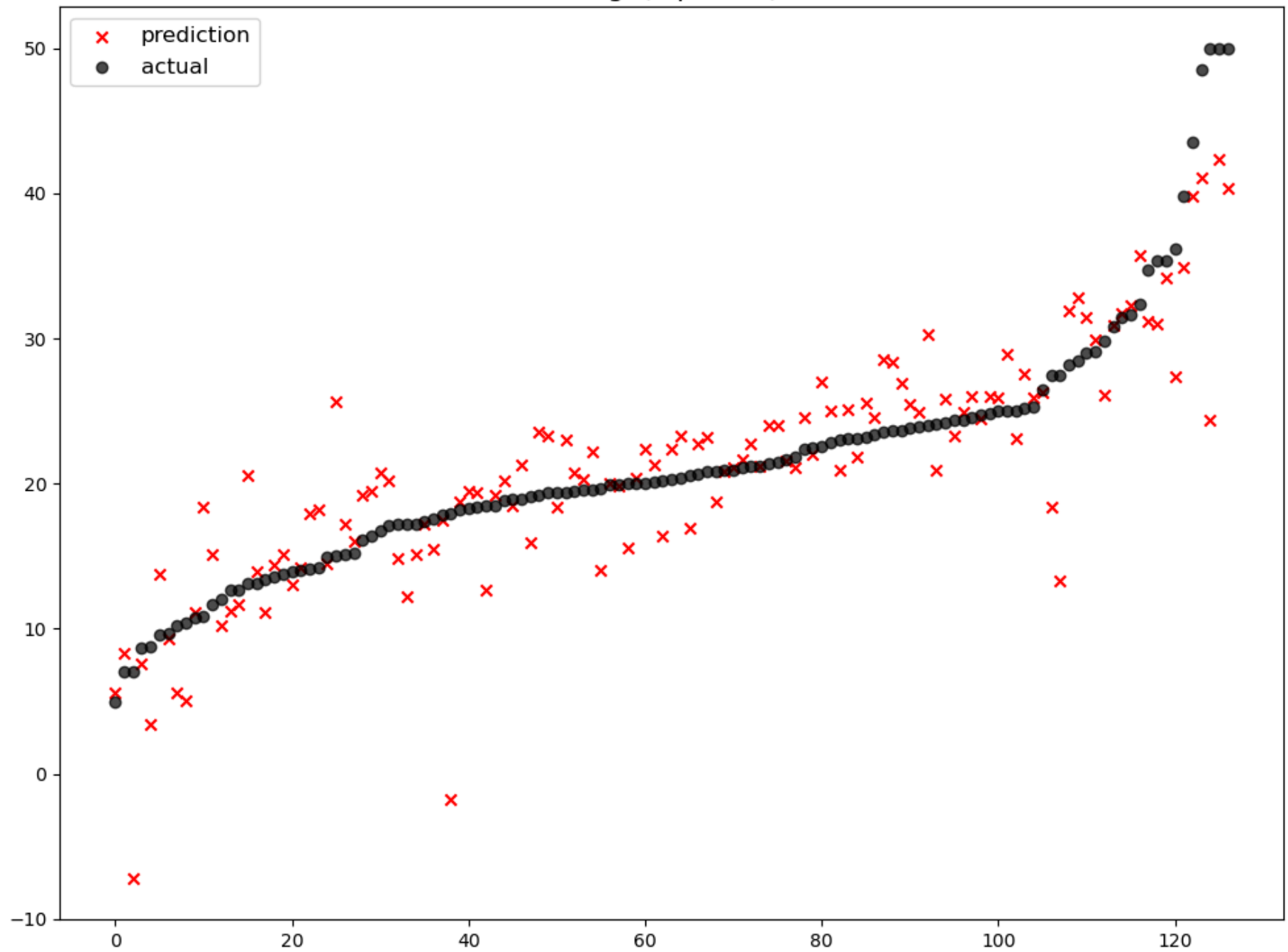


	model	mse
0	LinearRegression	22.098695

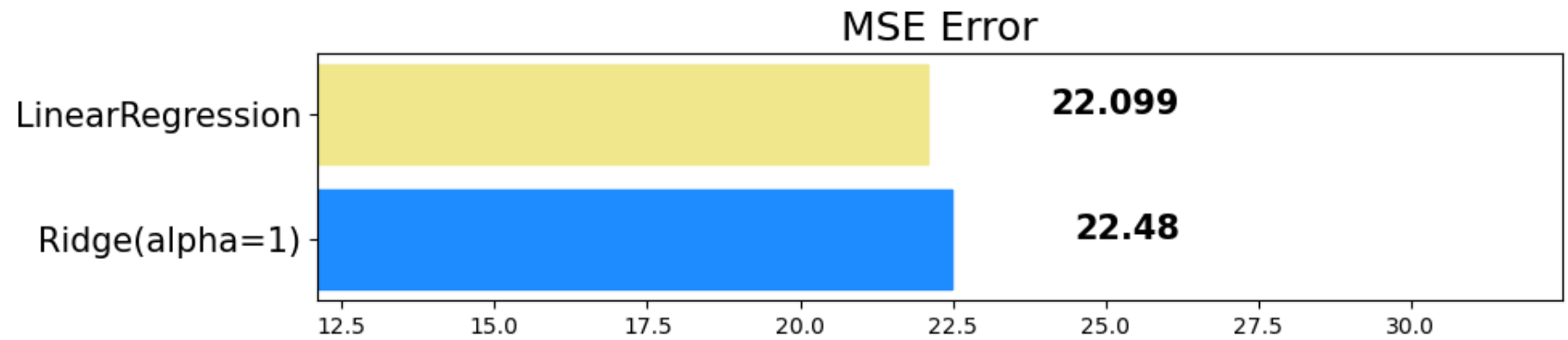


```
In [36]: ridge = Ridge(alpha=1)
ridge.fit(x_train, y_train)
pred = ridge.predict(x_test)
mse_eval('Ridge(alpha=1)', pred, y_test)
```

Ridge(alpha=1)

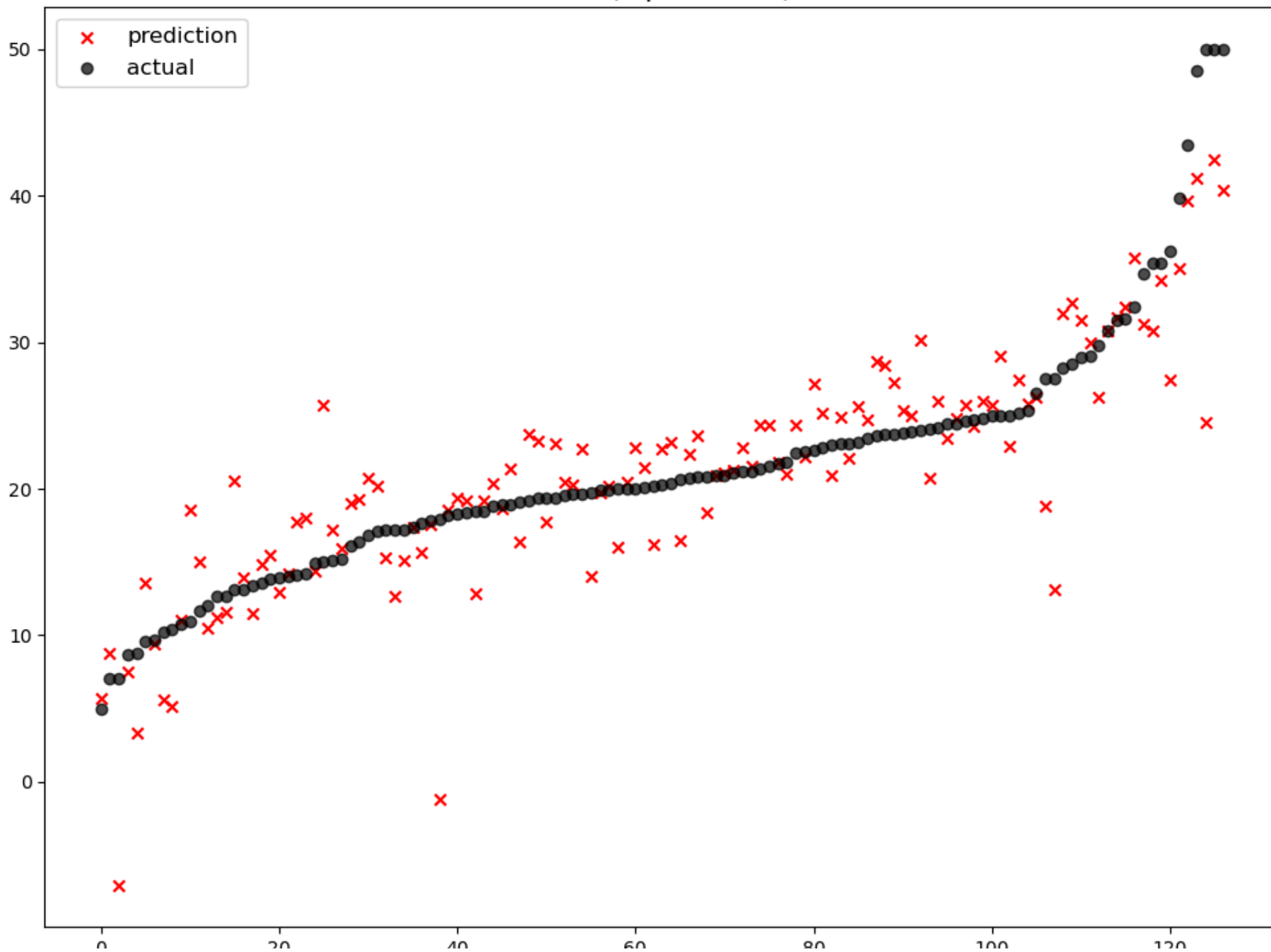


	model	mse
0	Ridge(alpha=1)	22.480476
1	LinearRegression	22.098695

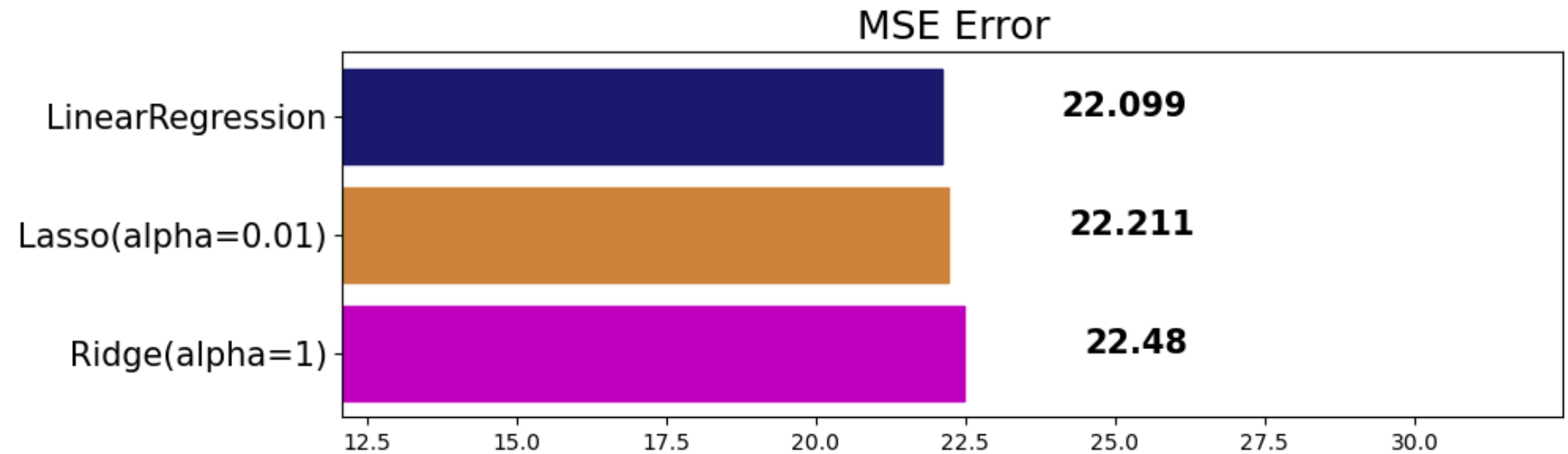


```
In [37]: lasso = Lasso(alpha=0.01)
lasso.fit(x_train, y_train)
pred = lasso.predict(x_test)
mse_eval('Lasso(alpha=0.01)', pred, y_test)
```

Lasso(alpha=0.01)

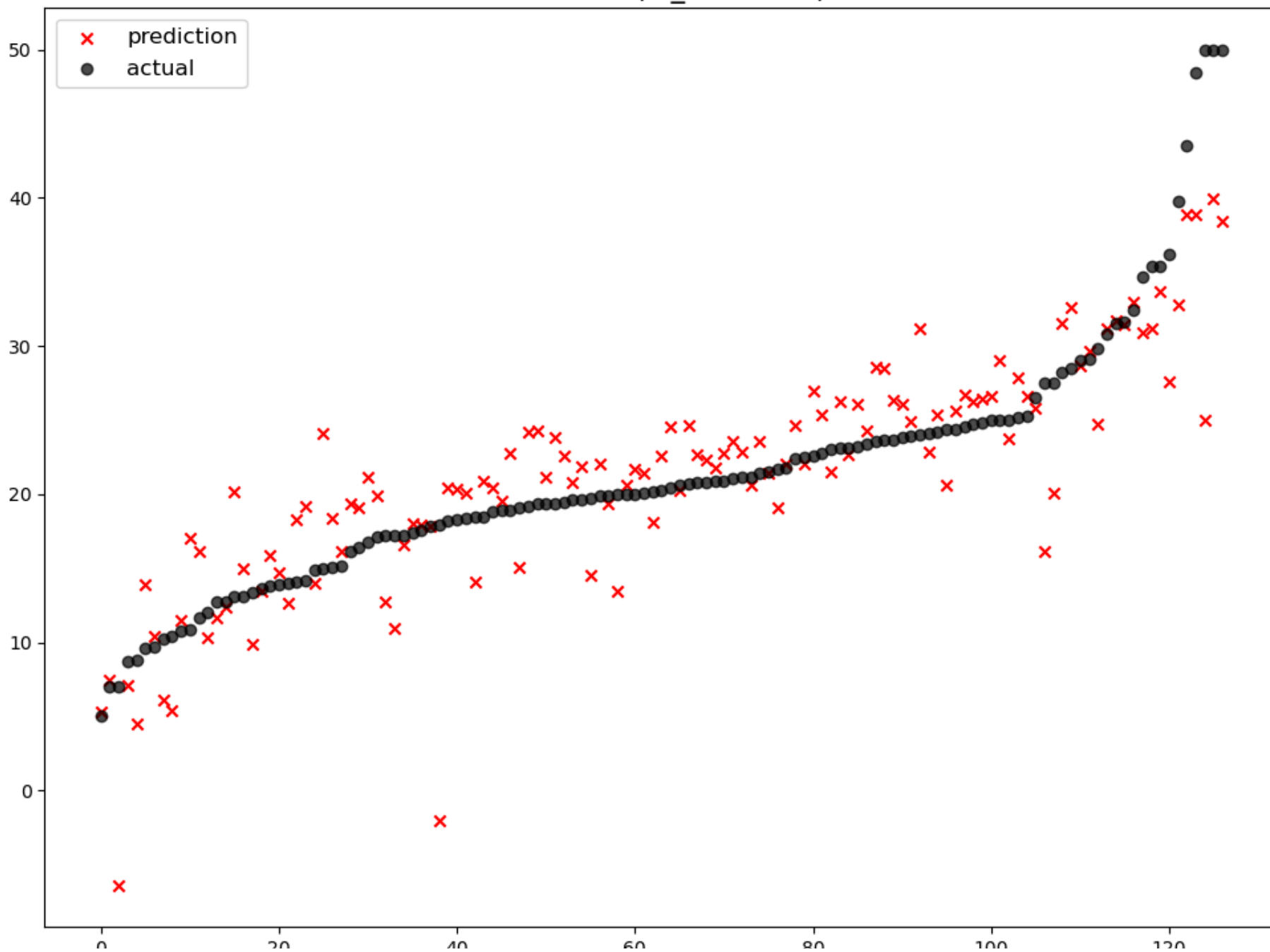


	model	mse
0	Ridge(alpha=1)	22.480476
1	Lasso(alpha=0.01)	22.210557
2	LinearRegression	22.098695

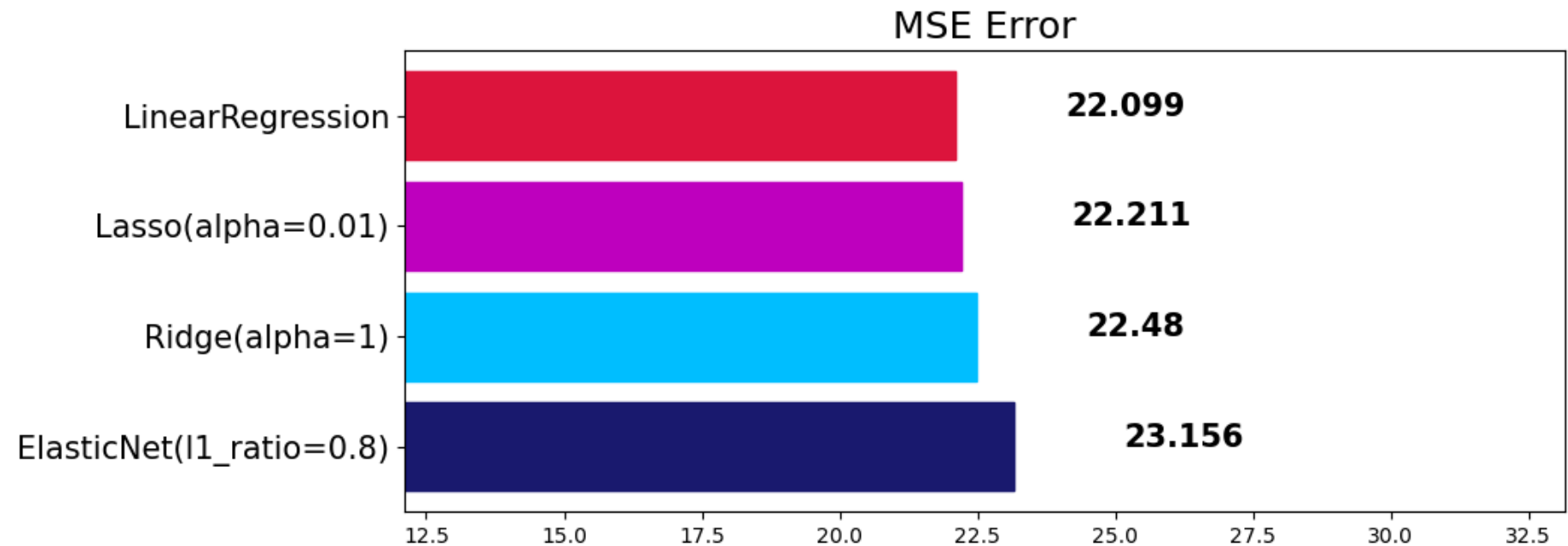


```
In [38]: elasticnet = ElasticNet(alpha=0.5, l1_ratio=0.8)
elasticnet.fit(x_train, y_train)
pred = elasticnet.predict(x_test)
mse_eval('ElasticNet(l1_ratio=0.8)', pred, y_test)
```

ElasticNet(l1_ratio=0.8)

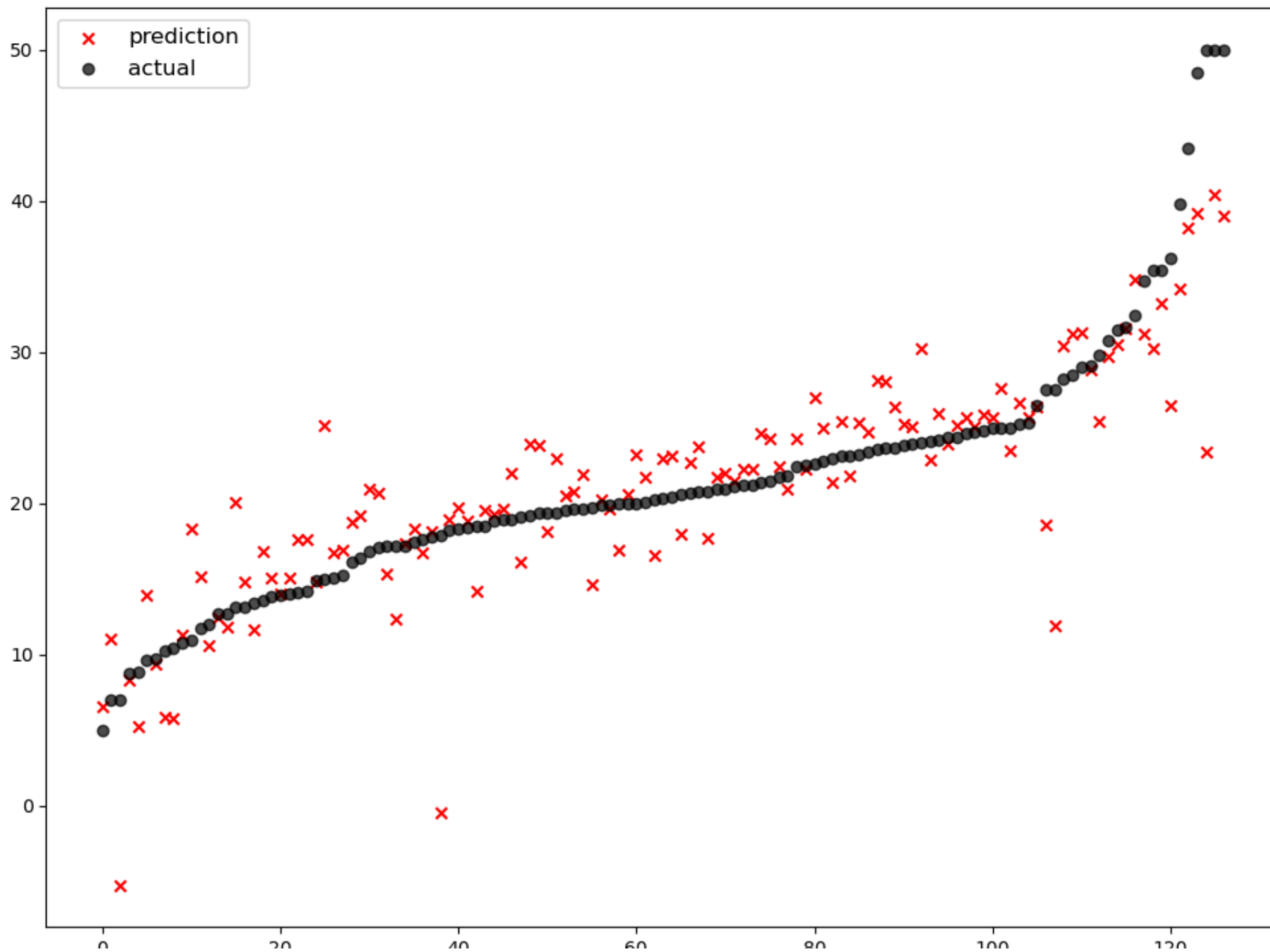


	model	mse
0	ElasticNet(l1_ratio=0.8)	23.156236
1	Ridge(alpha=1)	22.480476
2	Lasso(alpha=0.01)	22.210557
3	LinearRegression	22.098695

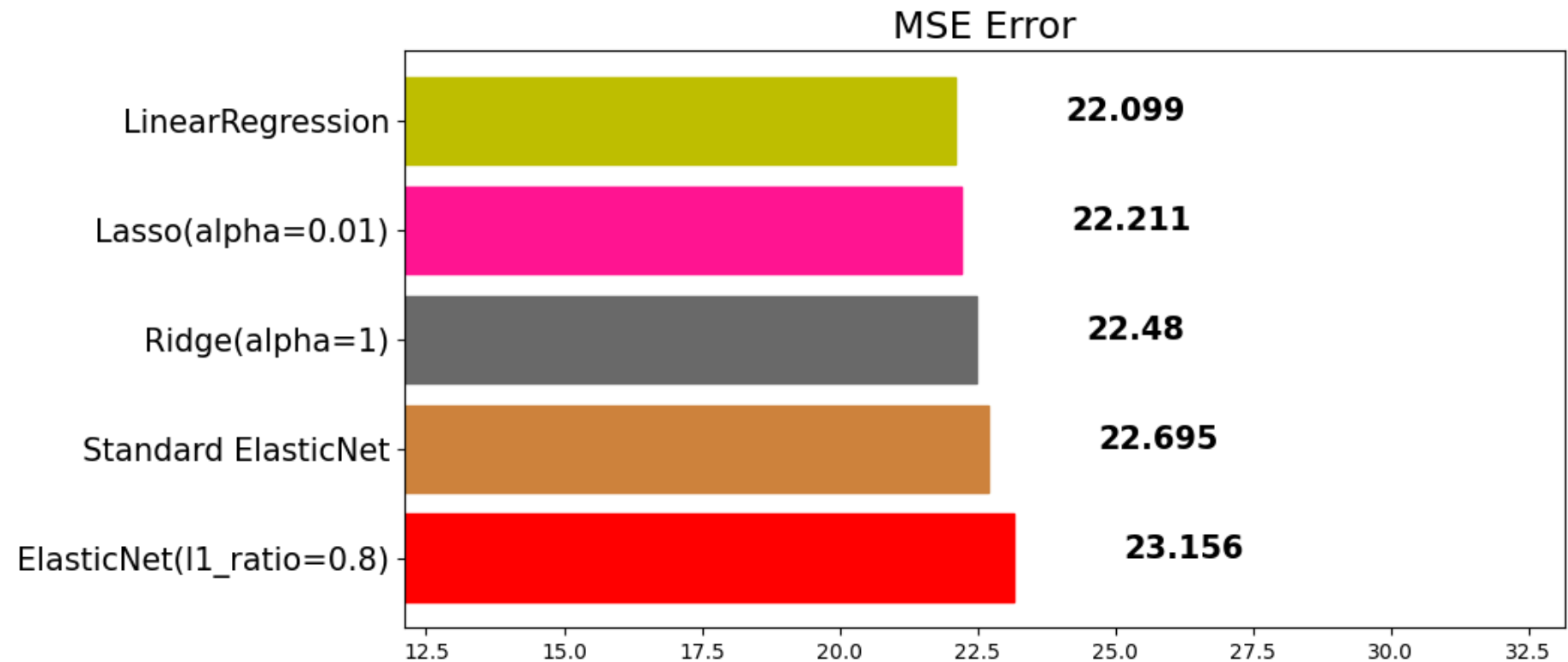


```
In [39]: elasticnet_pipeline = make_pipeline(
          StandardScaler(),
          ElasticNet(alpha=0.1, l1_ratio=0.2)
        )
elasticnet_pred = elasticnet_pipeline.fit(x_train, y_train).predict(x_test)
mse_eval('Standard ElasticNet', elasticnet_pred, y_test)
```

Standard ElasticNet



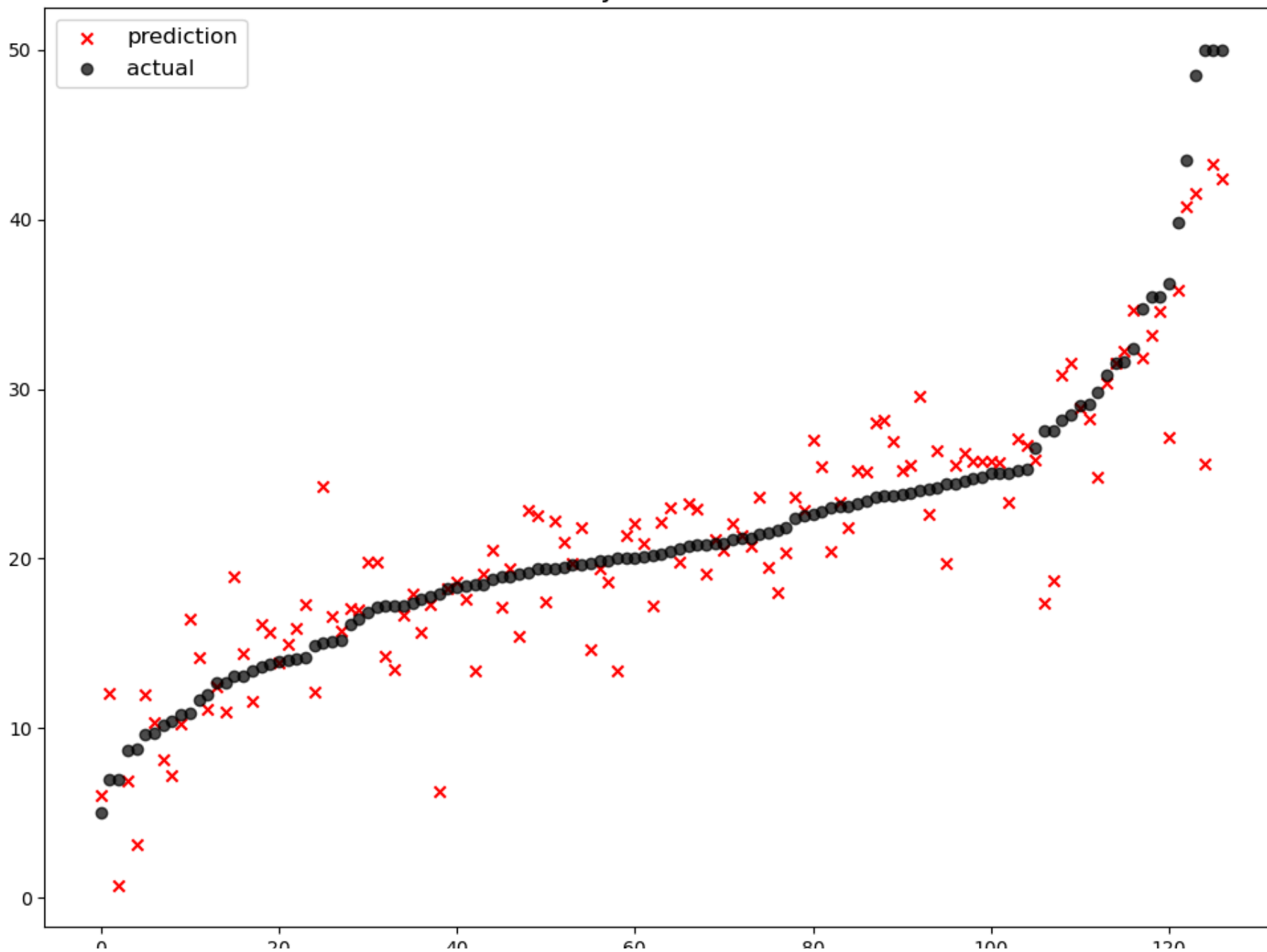
	model	mse
0	ElasticNet(l1_ratio=0.8)	23.156236
1	Standard ElasticNet	22.694970
2	Ridge(alpha=1)	22.480476
3	Lasso(alpha=0.01)	22.210557
4	LinearRegression	22.098695



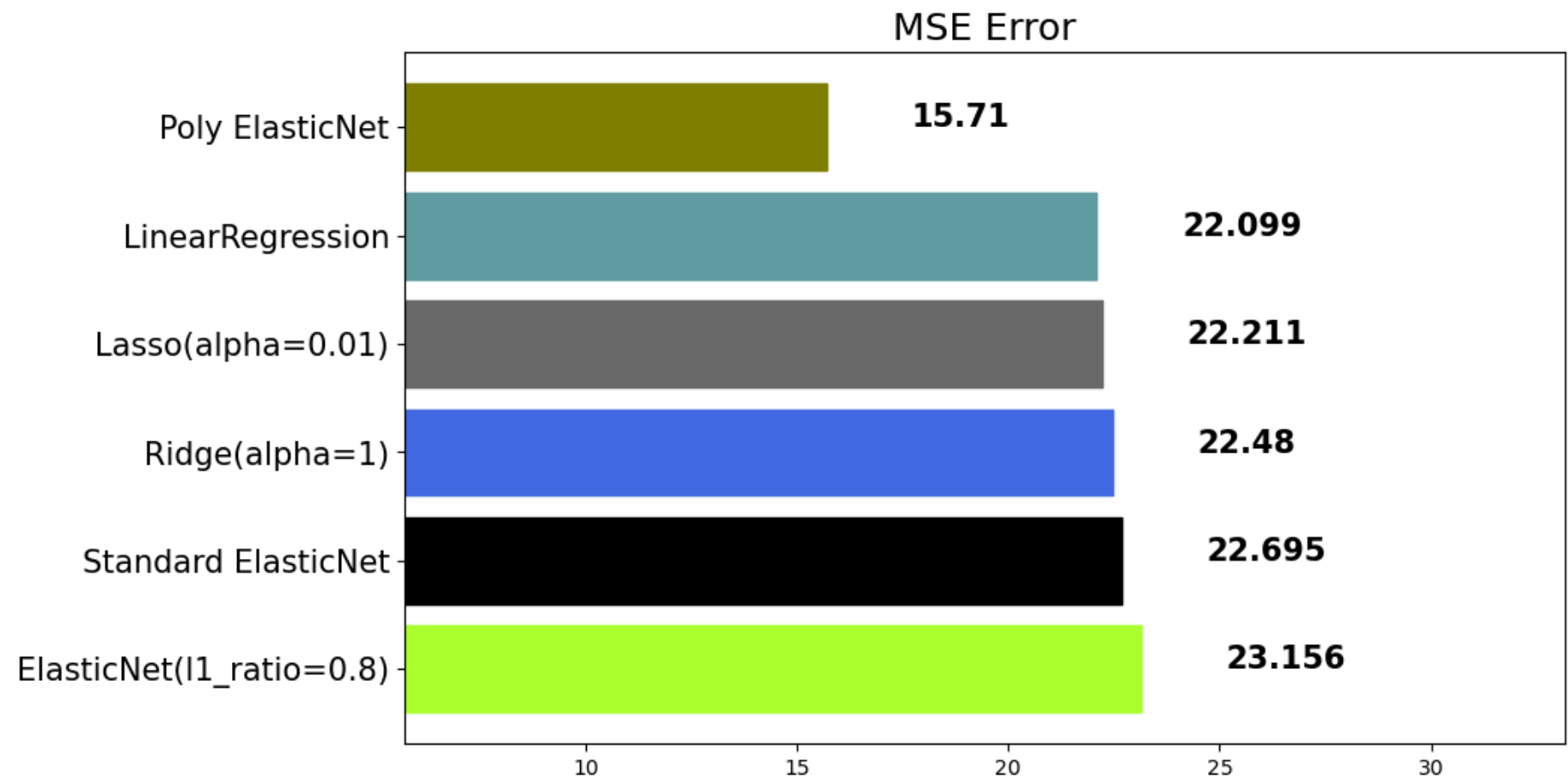
```
In [40]: poly_pipeline = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    StandardScaler(),
    ElasticNet(alpha=0.1, l1_ratio=0.2)
)
poly_pred = poly_pipeline.fit(x_train, y_train).predict(x_test)
mse_eval('Poly ElasticNet', poly_pred, y_test)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:697: ConvergenceWarning: Objective did not converge. You might want to increase the number of iterations, check the scale of the features or consider increasing regularisation. Duality gap: 2.782e+01, tolerance: 3.361e+00
  model = cd_fast.enet_coordinate_descent(
```

Poly ElasticNet



	model	mse
0	ElasticNet(l1_ratio=0.8)	23.156236
1	Standard ElasticNet	22.694970
2	Ridge(alpha=1)	22.480476
3	Lasso(alpha=0.01)	22.210557
4	LinearRegression	22.098695
5	Poly ElasticNet	15.710492



앙상블 (Ensemble)

[scikit-learn](#) [도큐먼트](#)

보팅 (Voting) - 회귀 (Regression)

Voting은 단어 뜻 그대로 **투표를 통해 결정하는 방식**입니다. Voting은 Bagging과 투표방식이라는 점에서 유사하지만, 다음과 같은 큰 차이점이 있습니다.

- Voting은 다른 알고리즘 model을 조합해서 사용합니다.
- Bagging은 같은 알고리즘 내에서 다른 sample 조합을 사용합니다.

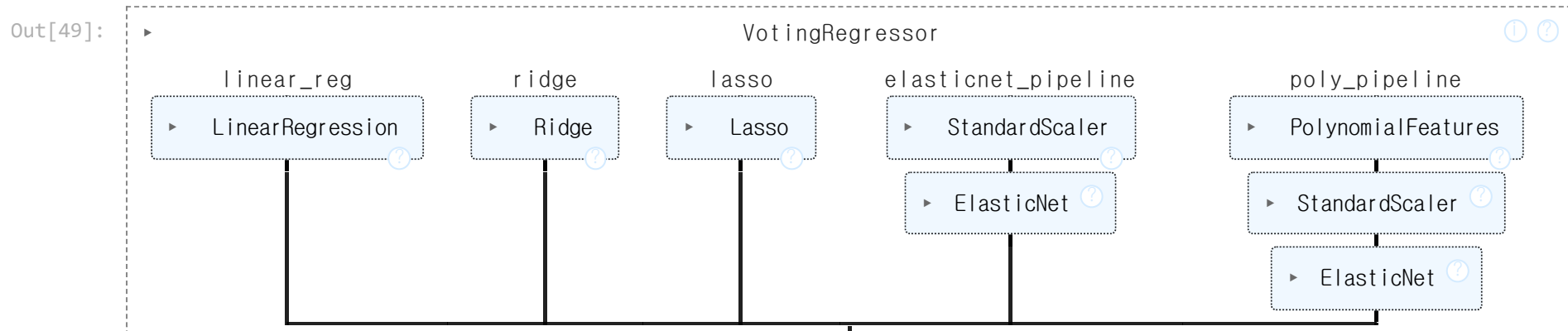
```
In [45]: from sklearn.ensemble import VotingRegressor, VotingClassifier
```

반드시, **Tuple 형태로 모델**을 정의해야 합니다.

```
In [47]: single_models = [  
    ('linear_reg', linear_reg),  
    ('ridge', ridge),  
    ('lasso', lasso),  
    ('elasticnet_pipeline', elasticnet_pipeline),  
    ('poly_pipeline', poly_pipeline)  
]
```

```
In [48]: voting_regressor = VotingRegressor(single_models, n_jobs=-1)
```

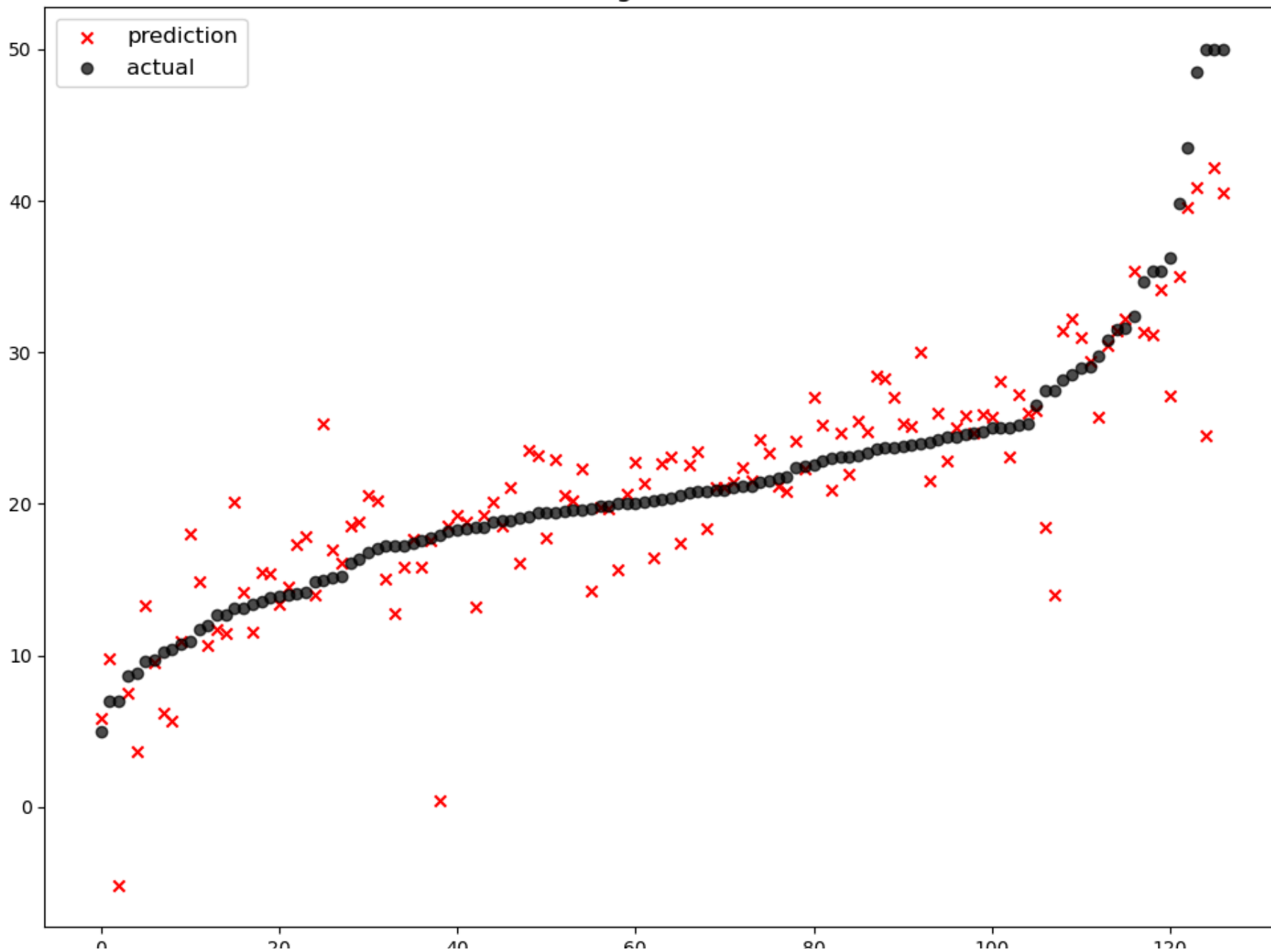
```
In [49]: voting_regressor.fit(x_train, y_train)
```



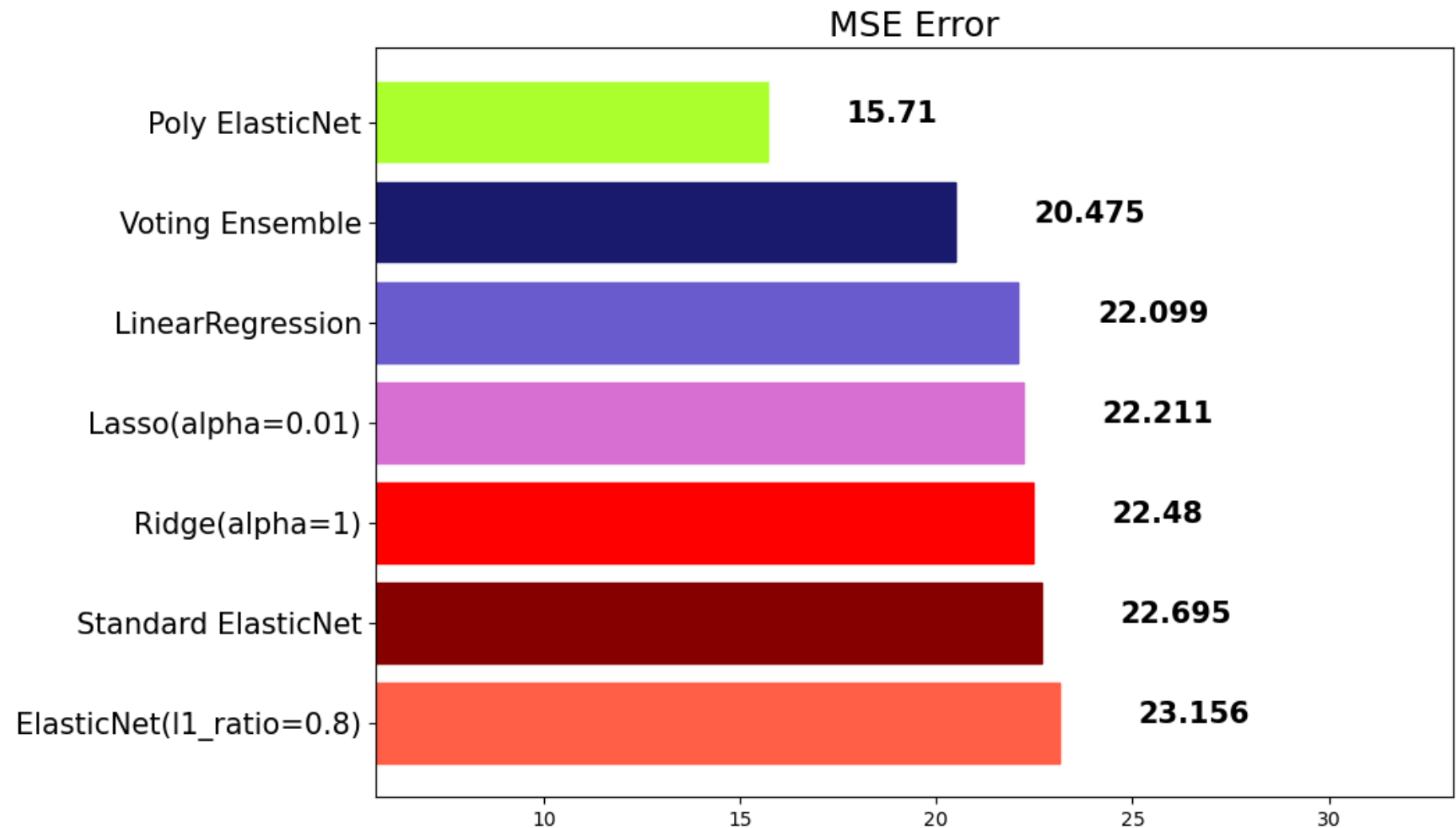
```
In [50]: voting_pred = voting_regressor.predict(x_test)
```

```
In [51]: mse_eval('Voting Ensemble', voting_pred, y_test)
```

Voting Ensemble



	model	mse
0	ElasticNet(l1_ratio=0.8)	23.156236
1	Standard ElasticNet	22.694970
2	Ridge(alpha=1)	22.480476
3	Lasso(alpha=0.01)	22.210557
4	LinearRegression	22.098695
5	Voting Ensemble	20.474757
6	Poly ElasticNet	15.710492



보팅 (Voting) - 분류 (Classification)

자세한 내용 보기

분류기 모델을 만들때, Voting 앙상블은 1가지의 **중요한 parameter**가 있습니다.

```
voting = {'hard', 'soft'}
```

hard로 설정한 경우

class를 0, 1로 분류 예측을 하는 이진 분류를 예로 들어 보겠습니다.

Hard Voting 방식에서는 결과 값에 대한 다수 class를 차용합니다.

classification을 예로 들어 보자면, 분류를 예측한 값이 1, 0, 0, 1, 1 이었다고 가정한다면 1이 3표, 0이 2표를 받았기 때문에 Hard Voting 방식에서는 1이 최종 값으로 예측을 하게 됩니다.

soft

soft vote 방식은 각각의 확률의 평균 값을 계산한다음에 가장 확률이 높은 값으로 확정짓게 됩니다.

가령 class 0이 나올 확률이 (0.4, 0.9, 0.9, 0.4, 0.4)이었고, class 1이 나올 확률이 (0.6, 0.1, 0.1, 0.6, 0.6) 이었다면,

- class 0이 나올 최종 확률은 $(0.4+0.9+0.9+0.4+0.4) / 5 = 0.44$,
- class 1이 나올 최종 확률은 $(0.6+0.1+0.1+0.6+0.6) / 5 = 0.4$

가 되기 때문에 앞선 Hard Vote의 결과와는 다른 결과 값이 최종 으로 선출되게 됩니다.

```
In [61]: from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression, RidgeClassifier
```

```
In [62]: models = [  
    ('Logi', LogisticRegression()),  
    ('ridge', RidgeClassifier())  
]
```

voting 옵션에 대하여 지정합니다.

```
In [64]: vc = VotingClassifier(models, voting='hard')
```

배깅(Bagging)

자세한 내용 보기

Bagging은 **Bootstrap Aggregating**의 줄임말입니다.

- Bootstrap = Sample(샘플) + Aggregating = 합산

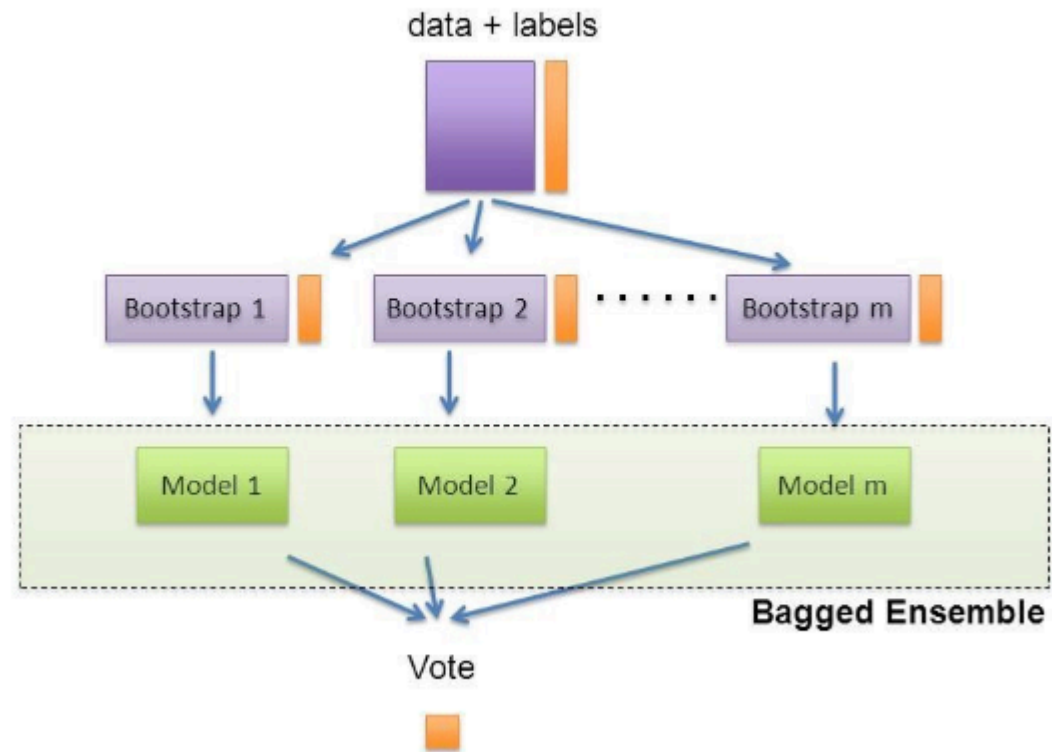
Bootstrap은 여러 개의 dataset을 중첩을 허용하게 하여 샘플링하여 분할하는 방식

데이터 셋의 구성이 [1, 2, 3, 4, 5]로 되어 있다면,

1. group 1 = [1, 2, 3]
2. group 2 = [1, 3, 4]
3. group 3 = [2, 3, 5]

```
In [69]: Image('https://teddylee777.github.io/images/2019-12-17/image-20191217015537872.png')
```

Out[69]: “Bagging” : Bootstrap **AGG**regat**ING**



Voting VS Bagging

- **Voting**은 여러 알고리즘의 조합에 대한 앙상블
- **Bagging**은 하나의 단일 알고리즘에 대하여 여러 개의 샘플 조합으로 앙상블

대표적인 Bagging 앙상블

1. RandomForest
2. Bagging

RandomForest

- DecisionTree(트리)기반 Bagging 앙상블
- 굉장히 인기있는 앙상블 모델
- 사용성이 쉽고, 성능도 우수함

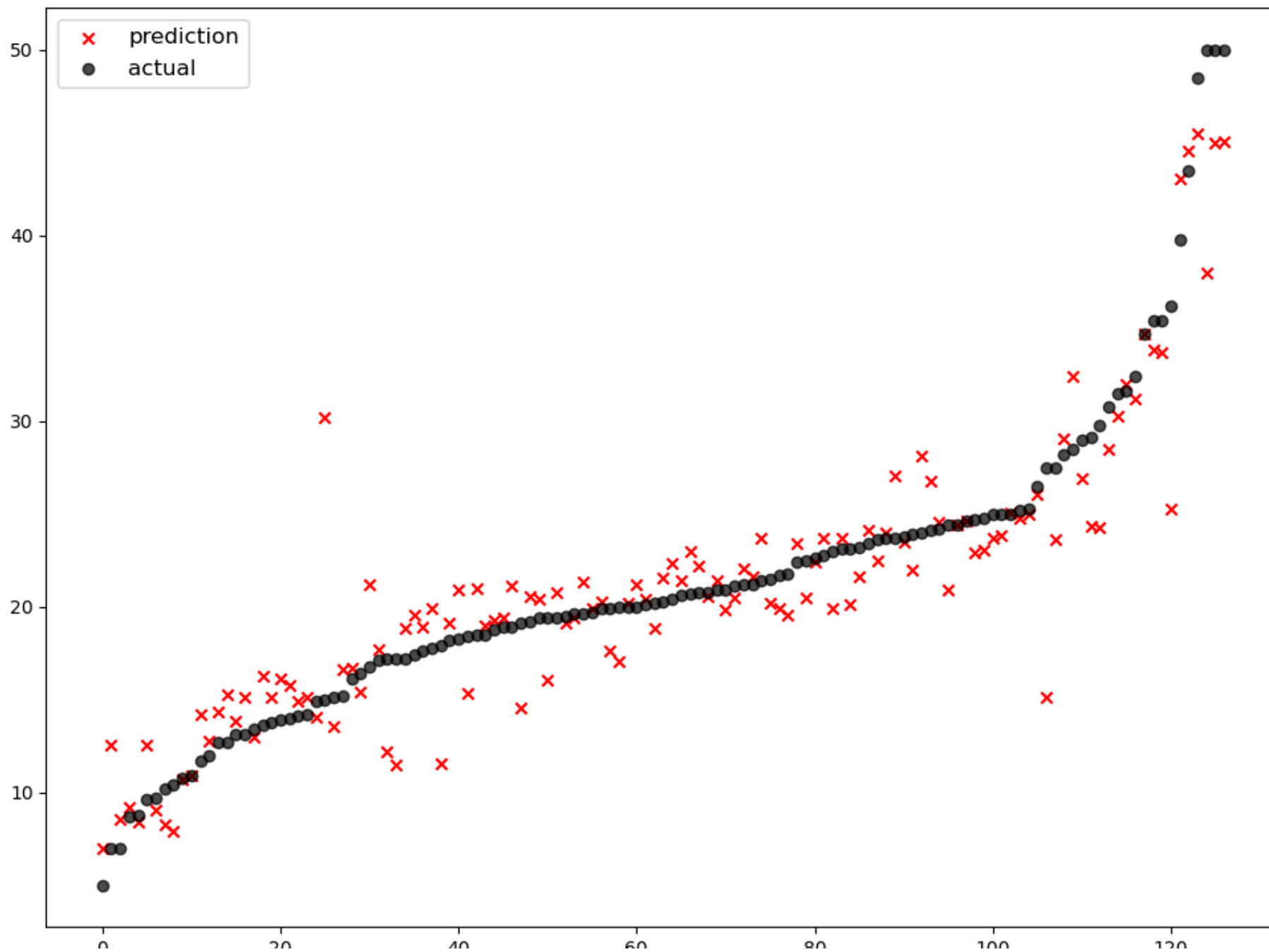
```
In [75]: from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
```

```
In [76]: rfr = RandomForestRegressor()  
rfr.fit(x_train, y_train)
```

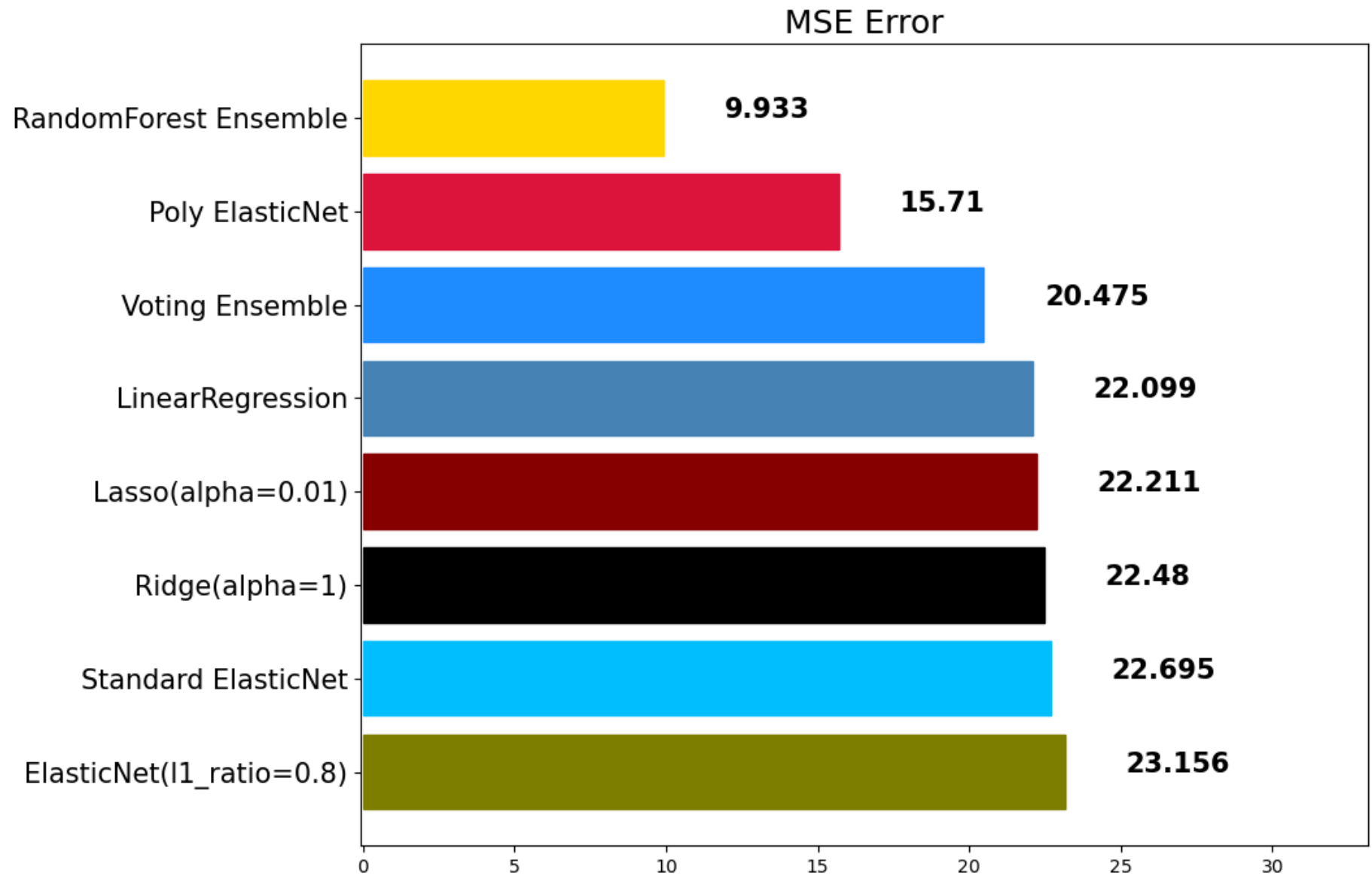
```
Out[76]: ▼ RandomForestRegressor ⓘ ?  
RandomForestRegressor()
```

```
In [77]: rfr_pred = rfr.predict(x_test)  
mse_eval('RandomForest Ensemble', rfr_pred, y_test)
```

RandomForest Ensemble



	0	20	40	60	80	100	120
	model		mse				
0	ElasticNet(l1_ratio=0.8)	23.156236					
1	Standard ElasticNet	22.694970					
2	Ridge(alpha=1)	22.480476					
3	Lasso(alpha=0.01)	22.210557					
4	LinearRegression	22.098695					
5	Voting Ensemble	20.474757					
6	Poly ElasticNet	15.710492					
7	RandomForest Ensemble	9.933307					



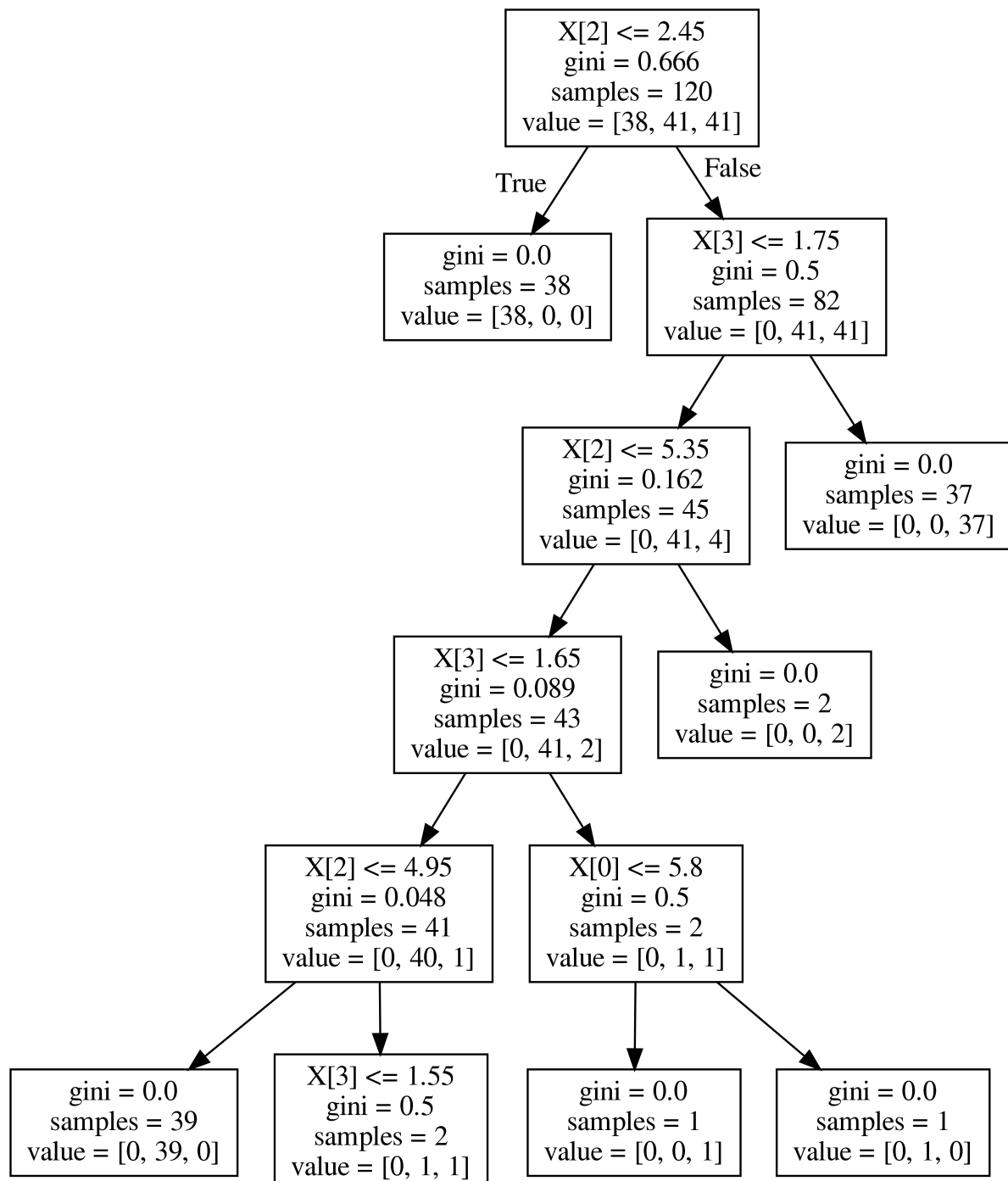
주요 Hyperparameter

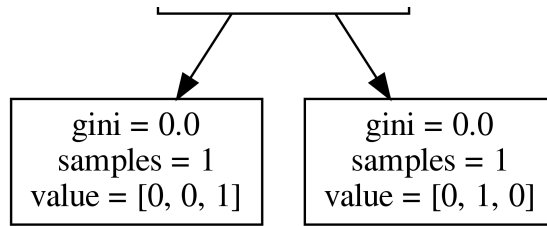
- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!

- n_jobs: CPU 사용 갯수
- max_depth: 깊어질 수 있는 최대 깊이. 과대적합 방지용
- n_estimators: 앙상블하는 트리의 갯수
- max_features: 최대로 사용할 feature의 갯수. 과대적합 방지용
- min_samples_split: 트리가 분할할 때 최소 샘플의 갯수. default=2. 과대적합 방지용

```
In [80]: Image('https://teddylee777.github.io/images/2020-01-09/decistion-tree.png', width=600)
```


Out[80]:

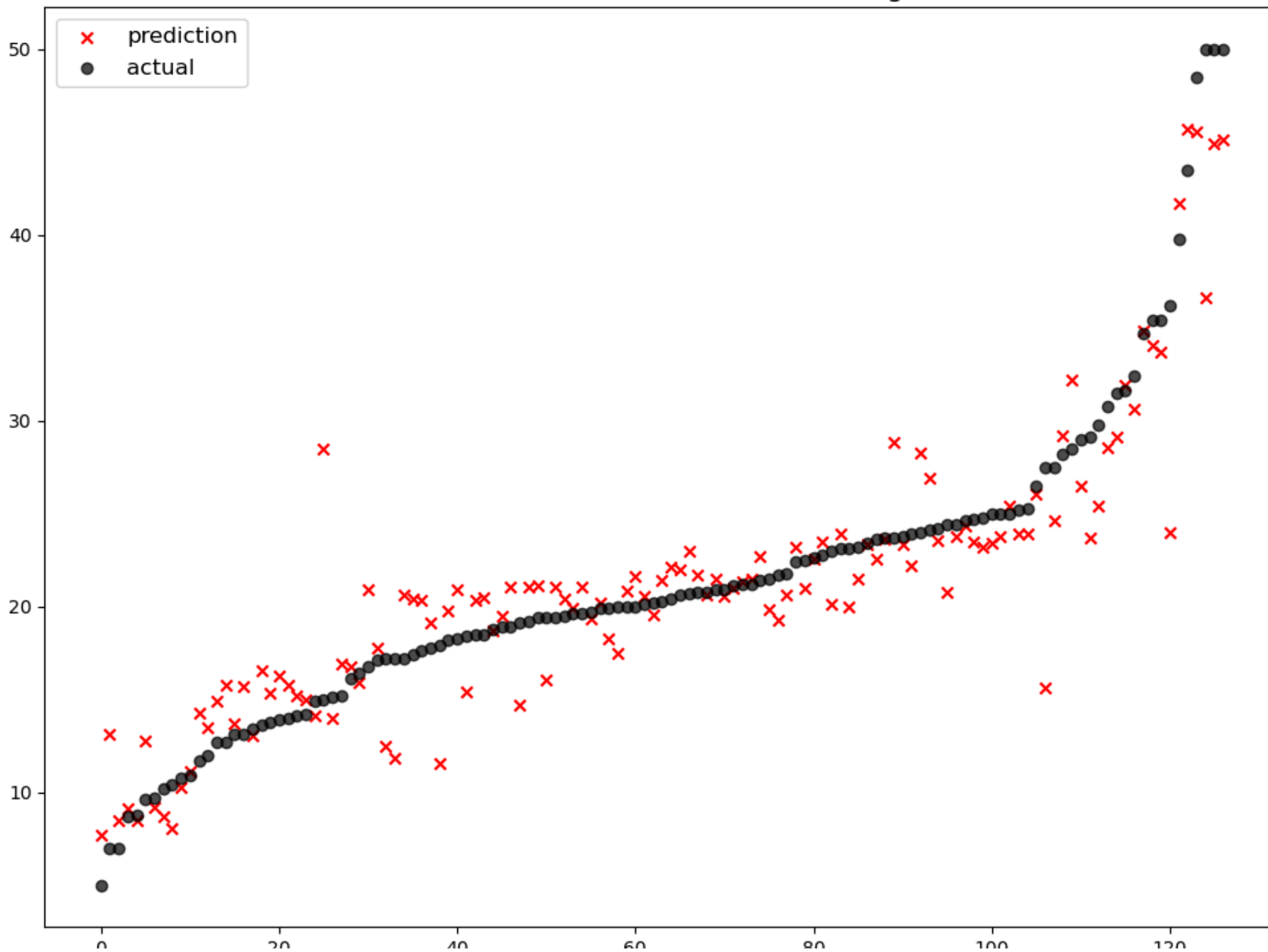




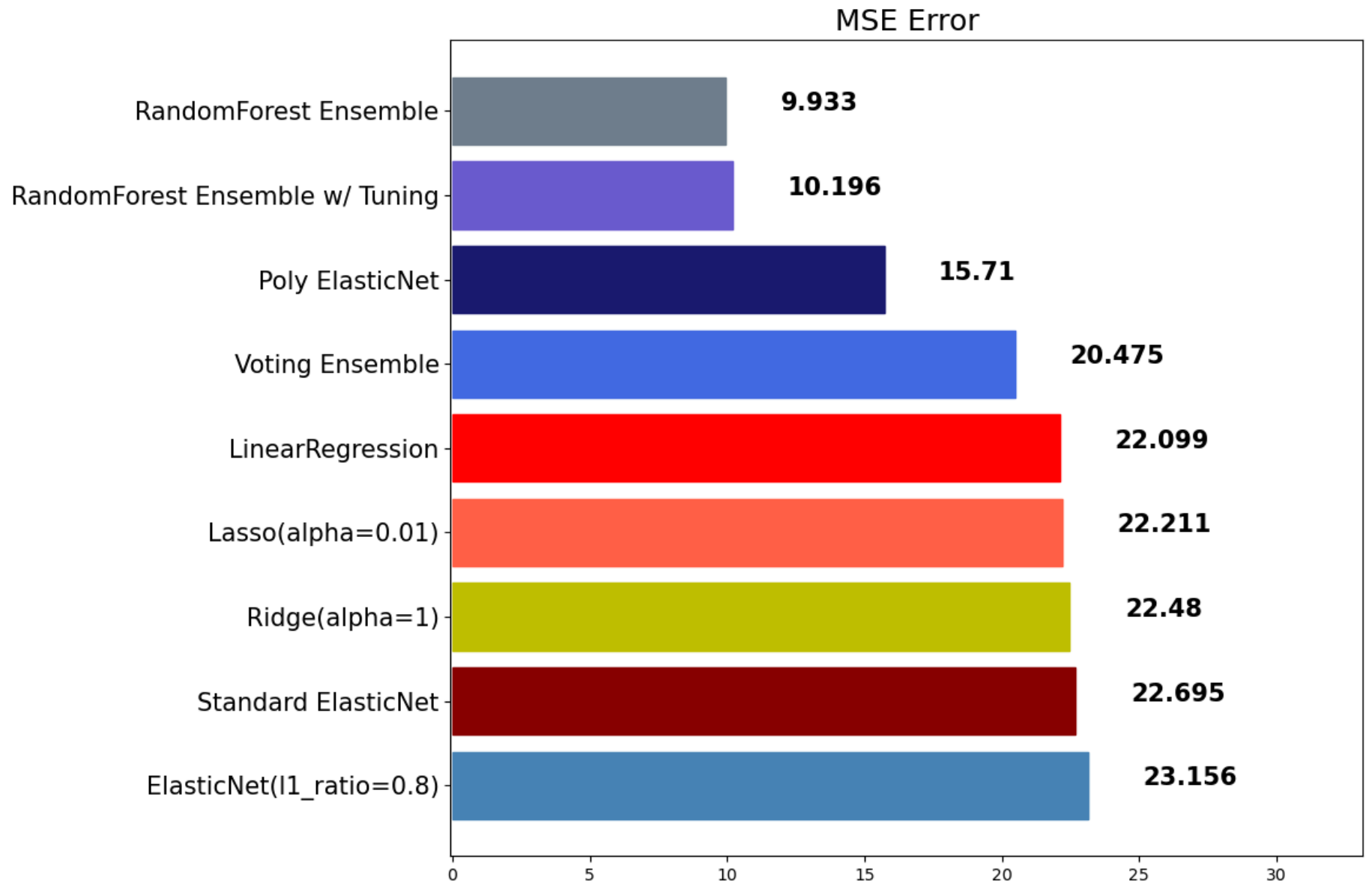
튜닝을 할 땐 반드시 `random_state` 값을 고정시킵니다!

```
In [82]: rfr = RandomForestRegressor(random_state=42, n_estimators=1000, max_depth=7, max_features=0.9)
rfr.fit(x_train, y_train)
rfr_pred = rfr.predict(x_test)
mse_eval('RandomForest Ensemble w/ Tuning', rfr_pred, y_test)
```

RandomForest Ensemble w/ Tuning



	0	20	40	60	80	100	120
	model		mse				
0	ElasticNet(l1_ratio=0.8)		23.156236				
1	Standard ElasticNet		22.694970				
2	Ridge(alpha=1)		22.480476				
3	Lasso(alpha=0.01)		22.210557				
4	LinearRegression		22.098695				
5	Voting Ensemble		20.474757				
6	Poly ElasticNet		15.710492				
7	RandomForest Ensemble w/ Tuning		10.196040				
8	RandomForest Ensemble		9.933307				



부스팅 (Boosting)

[자세한 내용 보기](#)

약한 학습기를 순차적으로 학습을 하되, 이전 학습에 대하여 잘못 예측된 데이터에 **가중치를 부여해 오차를 보완**해 나가는 방식입니다.

장점

- 성능이 매우 우수하다 (Lgbm, XGBoost)

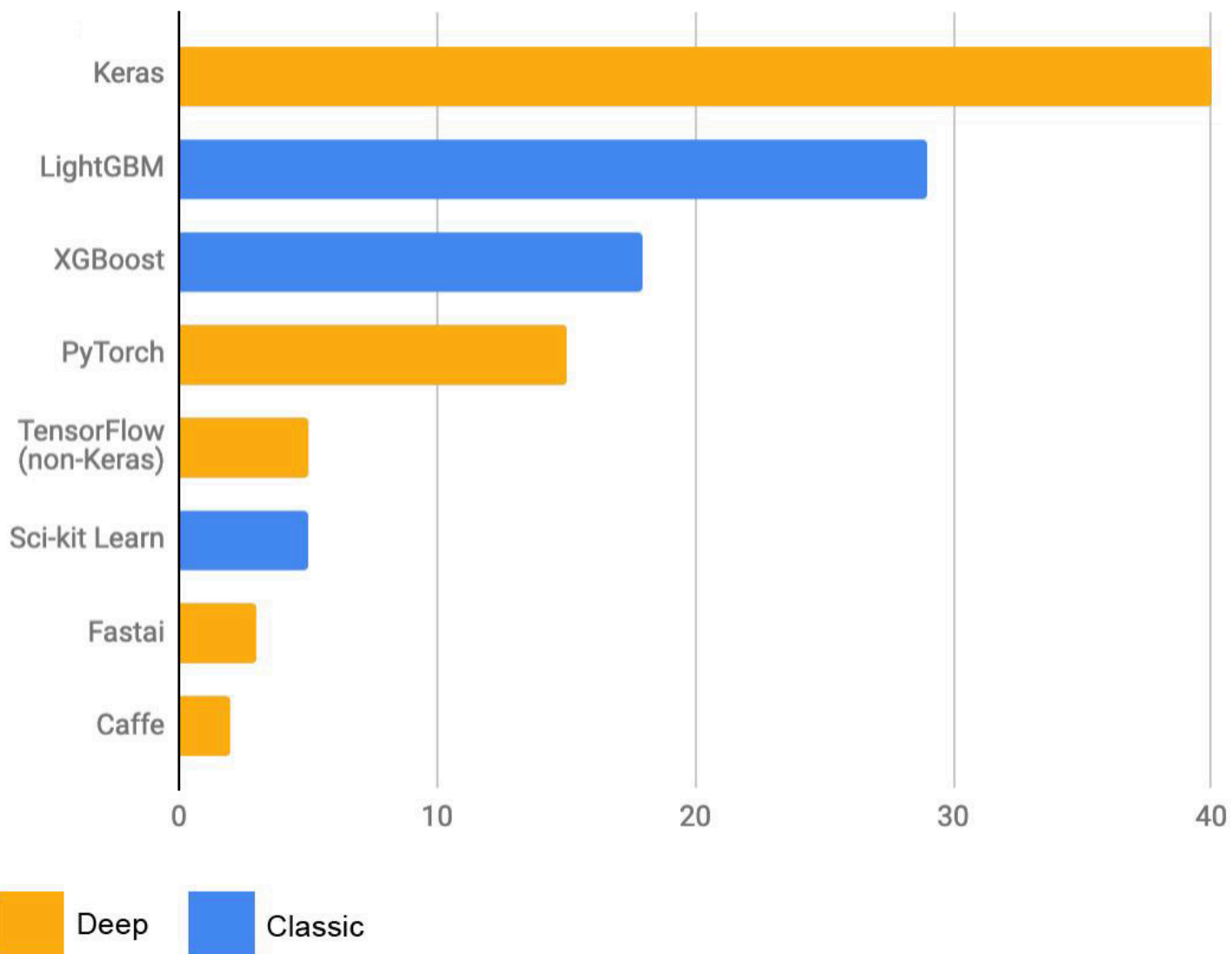
단점

- 부스팅 알고리즘의 특성상 계속 약점(오분류/잔차)을 보완하려고 하기 때문에 **잘못된 레이블링이나 아웃라이어에 필요 이상으로 민감**할 수 있다
- 다른 앙상블 대비 **학습 시간이 오래걸린다는 단점**이 존재

```
In [86]: Image('https://keras.io/img/graph-kaggle-1.jpeg', width=800)
```

Out[86]:

Primary ML software tool used by top-5 teams on Kaggle in each competition (n=120)



대표적인 Boosting 앙상블

1. AdaBoost
2. GradientBoost
3. LightGBM (LGBM)
4. XGBoost

GradientBoost

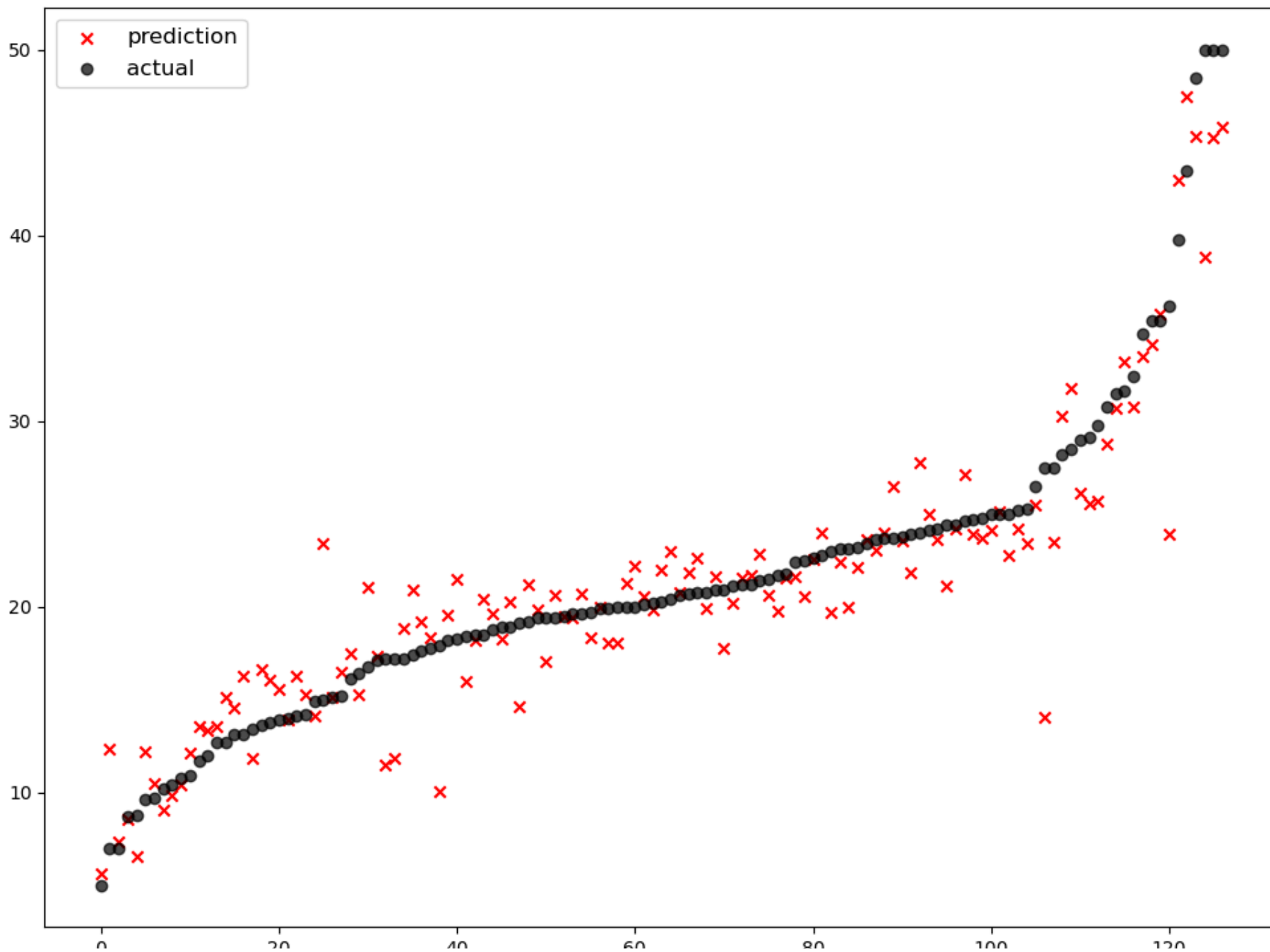
- 성능이 우수함
- 학습시간이 해도해도 너무 느리다

[사이킷런 공식문서](#)

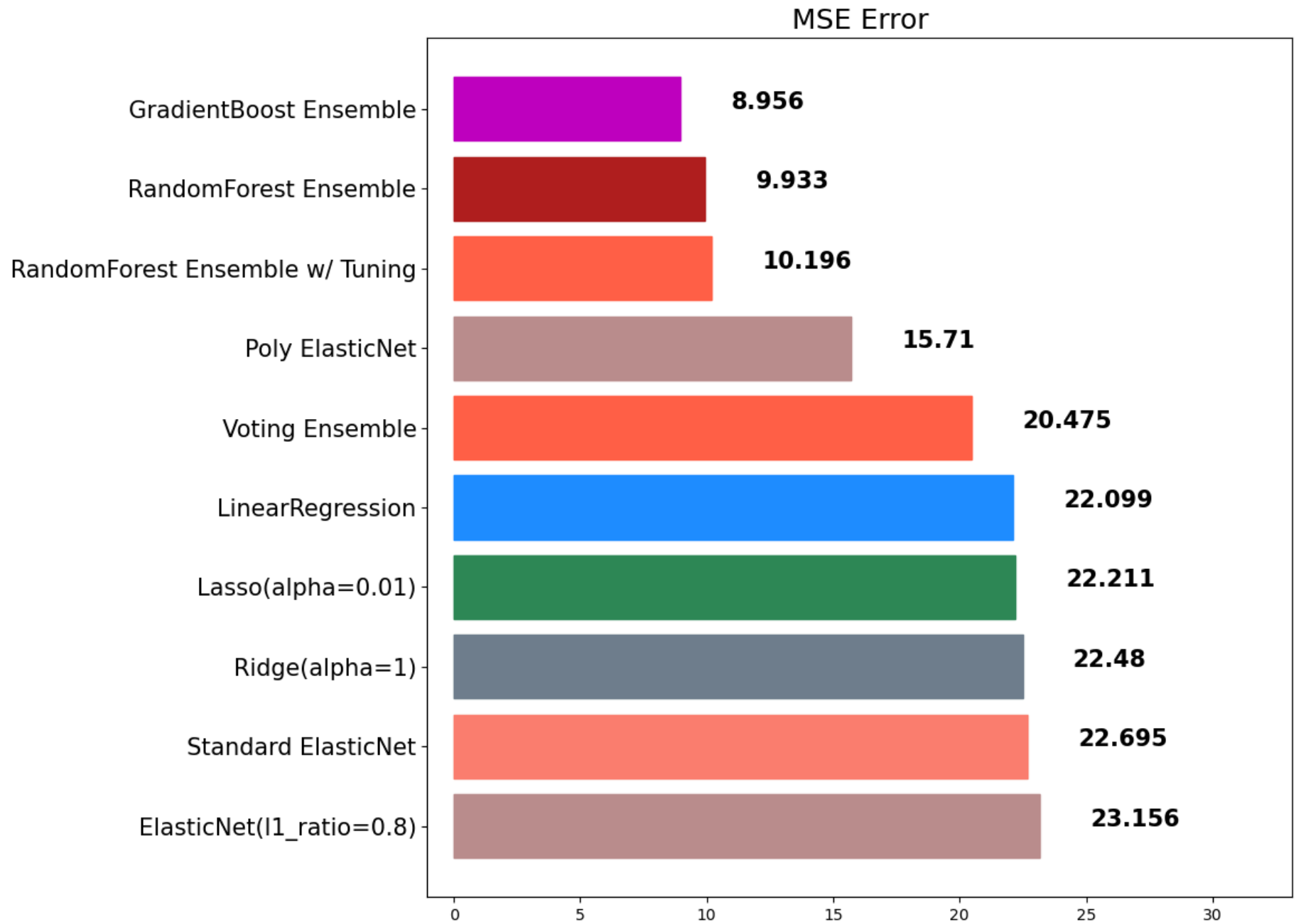
```
In [91]: from sklearn.ensemble import GradientBoostingRegressor, GradientBoostingClassifier
```

```
In [92]: gbr = GradientBoostingRegressor(random_state=42)
gbr.fit(x_train, y_train)
gbr_pred = gbr.predict(x_test)
mse_eval('GradientBoost Ensemble', gbr_pred, y_test)
```


GradientBoost Ensemble



	0	20	40	60	80	100	120
	model		mse				
0	ElasticNet(l1_ratio=0.8)		23.156236				
1	Standard ElasticNet		22.694970				
2	Ridge(alpha=1)		22.480476				
3	Lasso(alpha=0.01)		22.210557				
4	LinearRegression		22.098695				
5	Voting Ensemble		20.474757				
6	Poly ElasticNet		15.710492				
7	RandomForest Ensemble w/ Tuning		10.196040				
8	RandomForest Ensemble		9.933307				
9	GradientBoost Ensemble		8.956141				

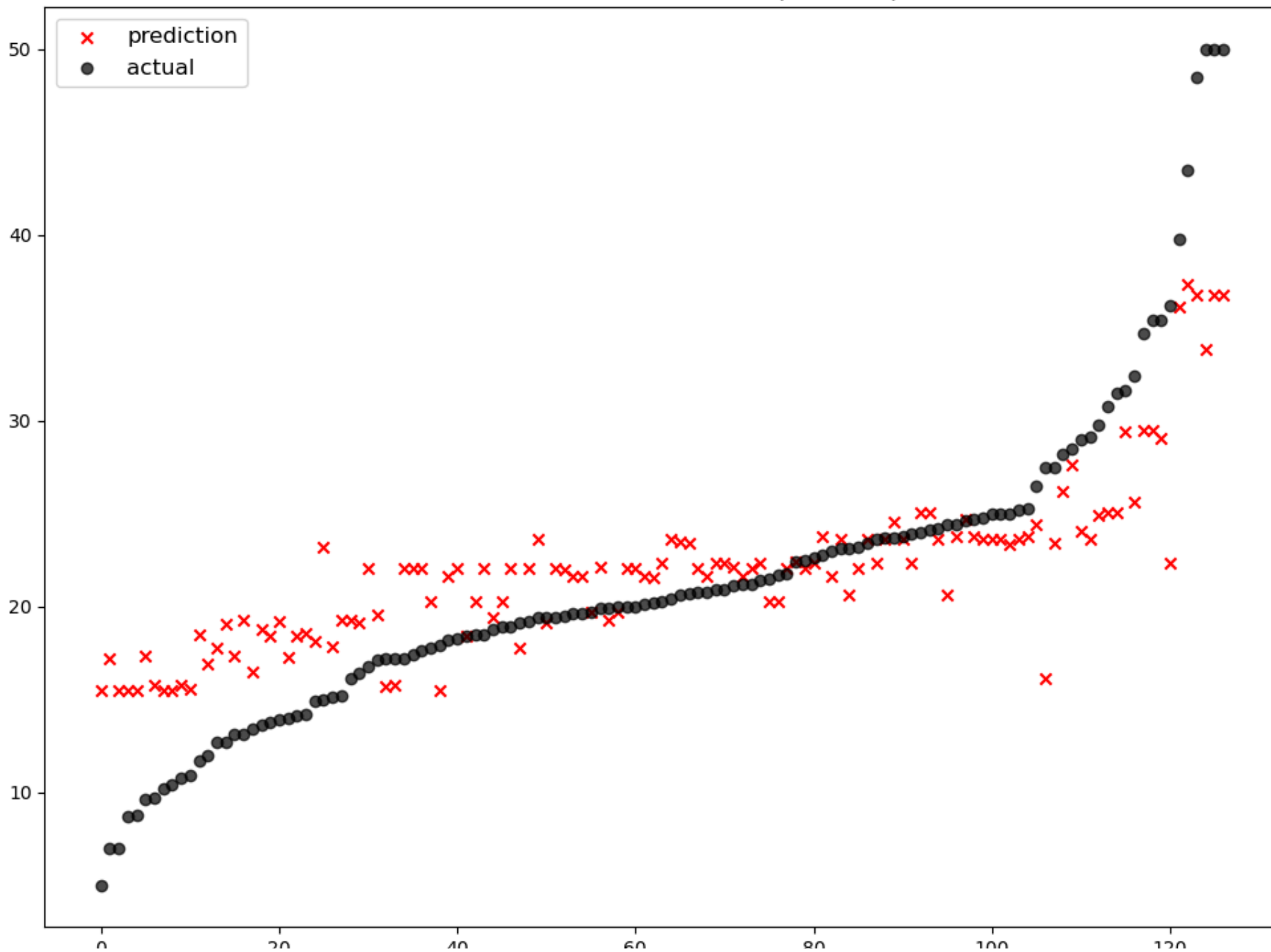


주요 Hyperparameter

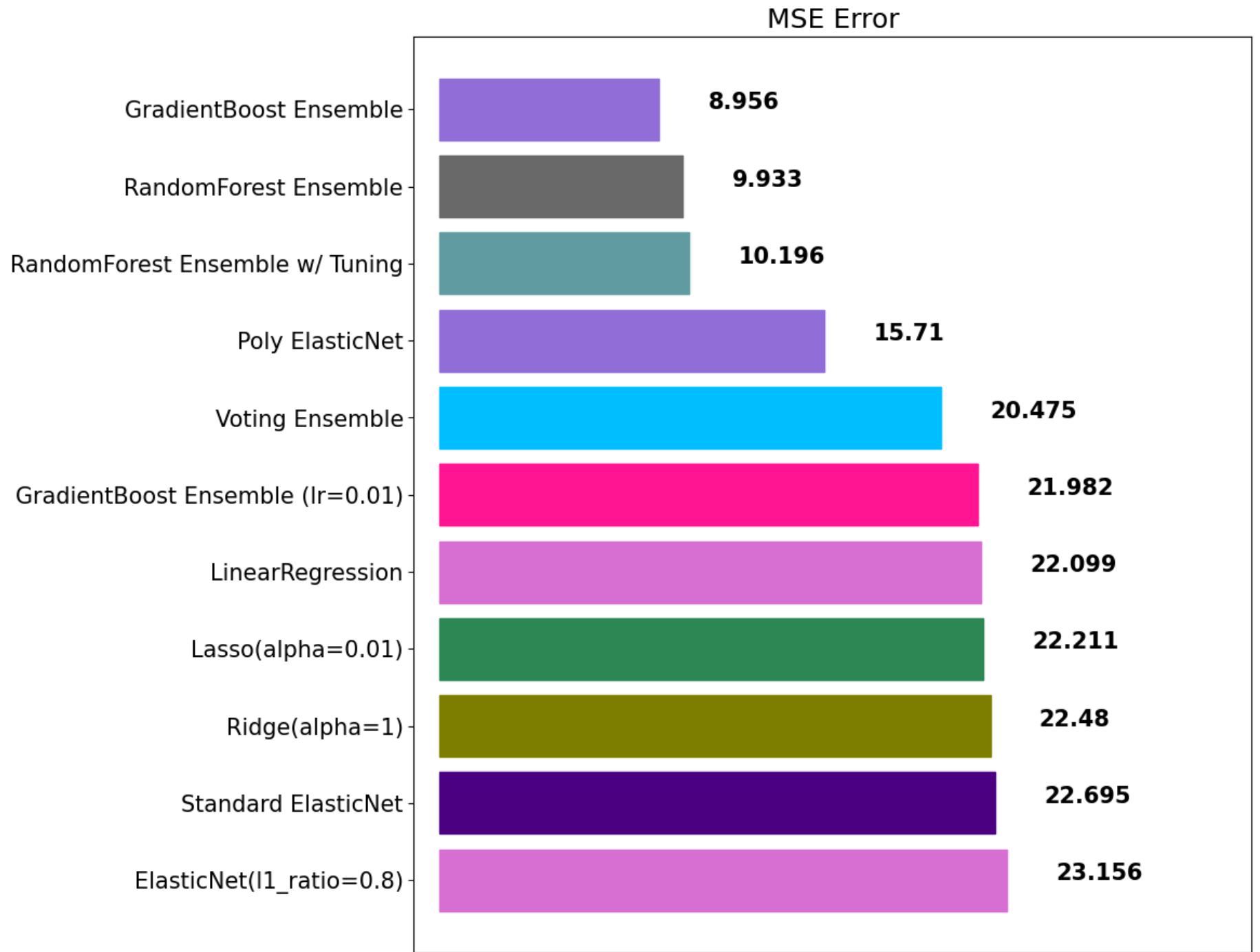
- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!
- n_jobs: CPU 사용 갯수
- learning_rate: 학습율. 너무 큰 학습율은 성능을 떨어뜨리고, 너무 작은 학습율은 학습이 느리다. 적절한 값을 찾아야함. n_estimators와 같이 튜닝. default=0.1
- n_estimators: 부스팅 스테이지 수. (랜덤포레스트 트리의 갯수 설정과 비슷한 개념). default=100
- subsample: 샘플 사용 비율 (max_features와 비슷한 개념). 과대적합 방지용
- min_samples_split: 노드 분할시 최소 샘플의 갯수. default=2. 과대적합 방지용

```
In [94]: gbr = GradientBoostingRegressor(random_state=42, learning_rate=0.01)
gbr.fit(x_train, y_train)
gbr_pred = gbr.predict(x_test)
mse_eval('GradientBoost Ensemble (lr=0.01)', gbr_pred, y_test)
```

GradientBoost Ensemble (lr=0.01)



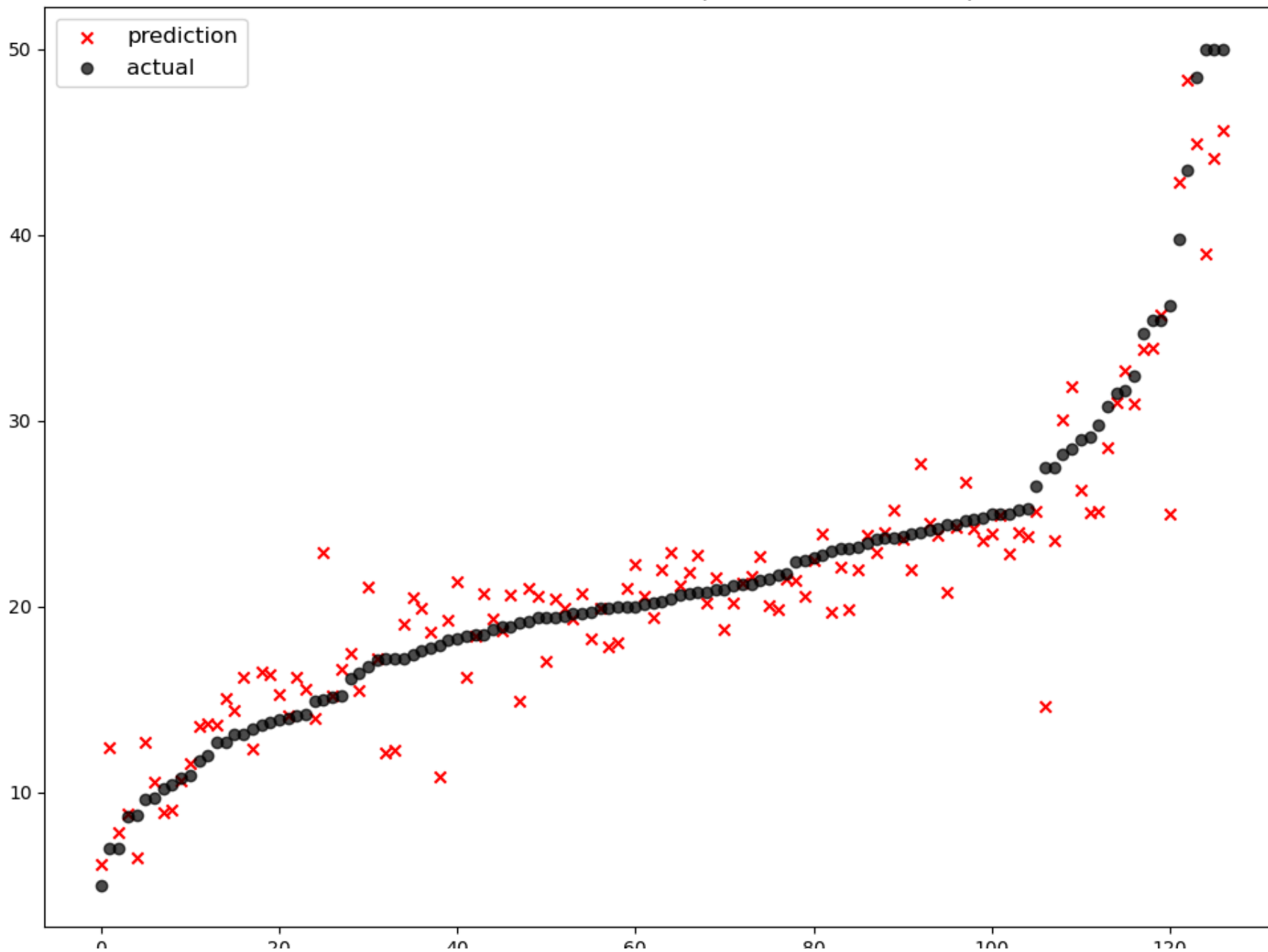
	0	20	40	60	80	100	120
		model		mse			
0		ElasticNet(l1_ratio=0.8)		23.156236			
1		Standard ElasticNet		22.694970			
2		Ridge(alpha=1)		22.480476			
3		Lasso(alpha=0.01)		22.210557			
4		LinearRegression		22.098695			
5	GradientBoost Ensemble (lr=0.01)			21.982151			
6		Voting Ensemble		20.474757			
7		Poly ElasticNet		15.710492			
8	RandomForest Ensemble w/ Tuning			10.196040			
9		RandomForest Ensemble		9.933307			
10		GradientBoost Ensemble		8.956141			



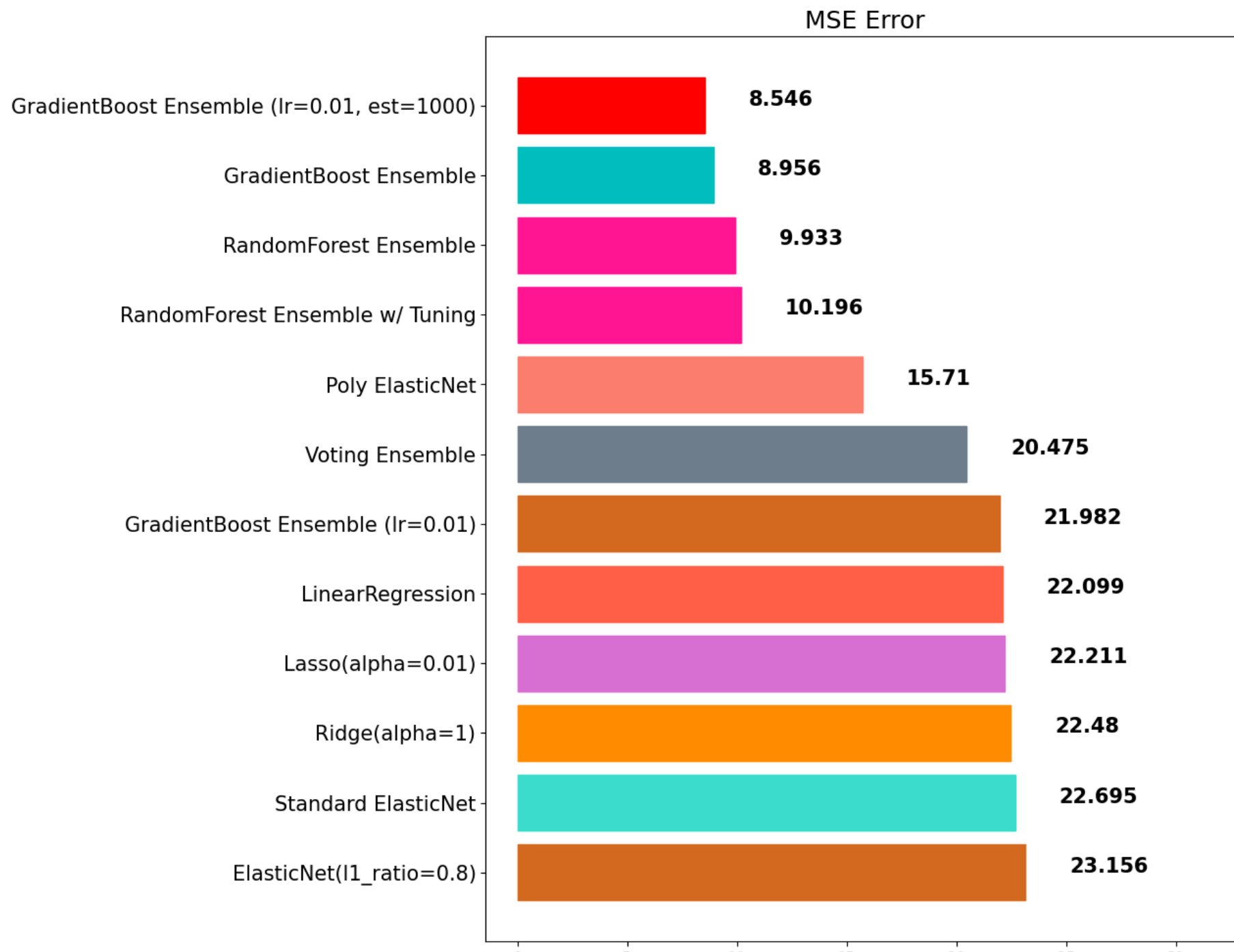
0 5 10 15 20 25 30

```
In [95]: gbr = GradientBoostingRegressor(random_state=42, learning_rate=0.01, n_estimators=1000)
gbr.fit(x_train, y_train)
gbr_pred = gbr.predict(x_test)
mse_eval('GradientBoost Ensemble (lr=0.01, est=1000)', gbr_pred, y_test)
```


GradientBoost Ensemble (lr=0.01, est=1000)



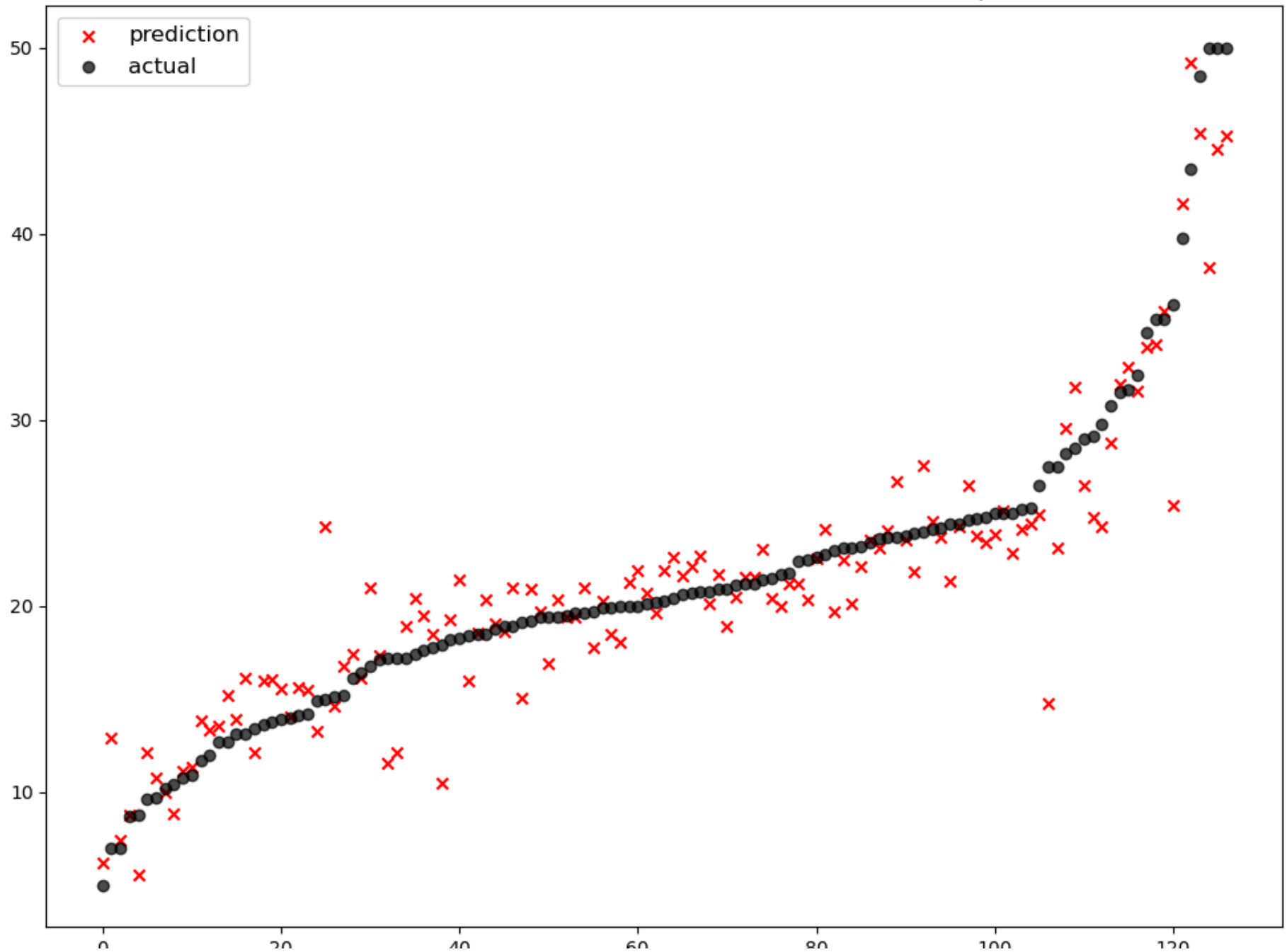
	model	mse
0	ElasticNet(l1_ratio=0.8)	23.156236
1	Standard ElasticNet	22.694970
2	Ridge(alpha=1)	22.480476
3	Lasso(alpha=0.01)	22.210557
4	LinearRegression	22.098695
5	GradientBoost Ensemble (lr=0.01)	21.982151
6	Voting Ensemble	20.474757
7	Poly ElasticNet	15.710492
8	RandomForest Ensemble w/ Tuning	10.196040
9	RandomForest Ensemble	9.933307
10	GradientBoost Ensemble	8.956141
11	GradientBoost Ensemble (lr=0.01, est=1000)	8.546408



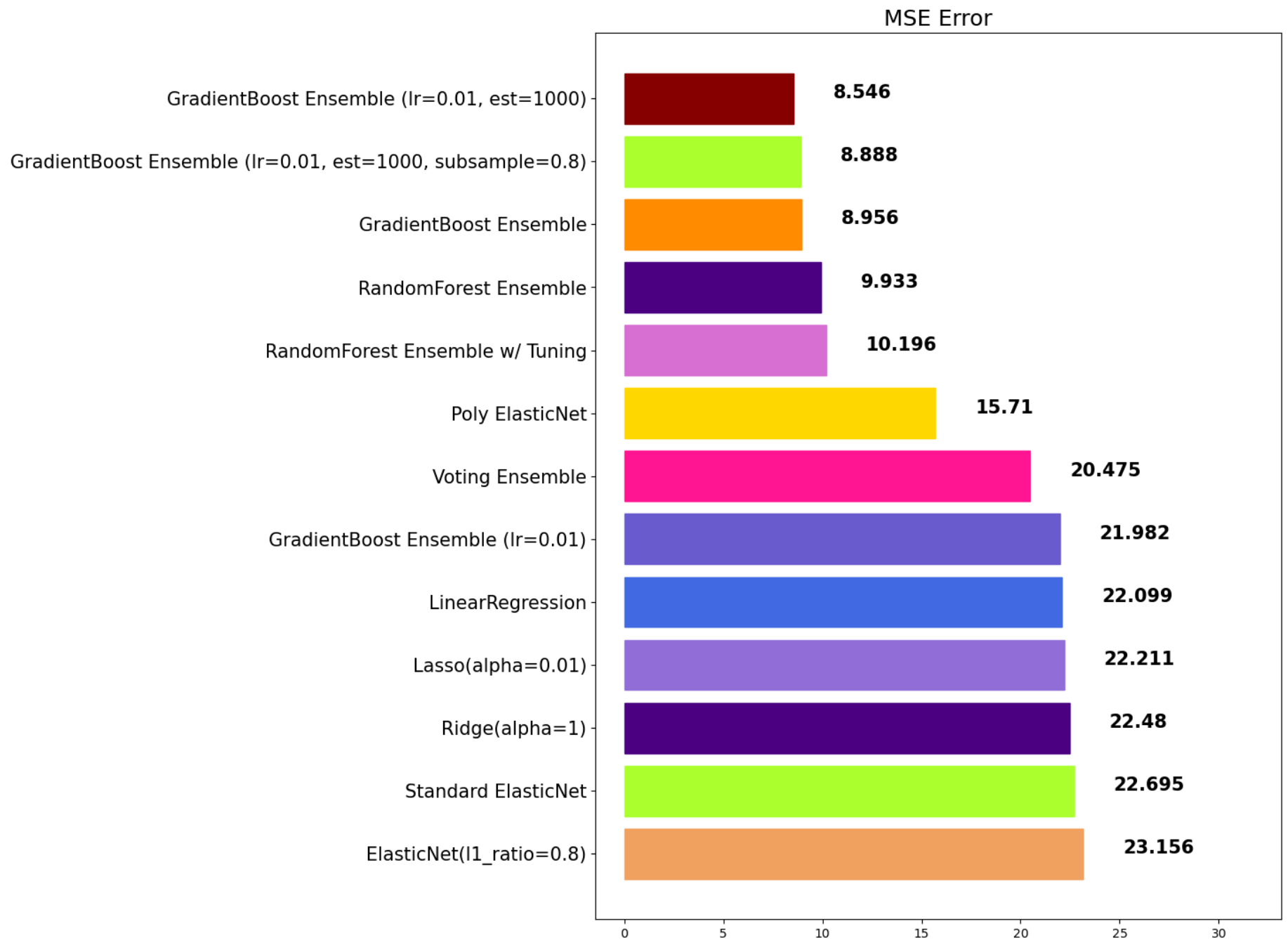
0 5 10 15 20 25 30

```
In [96]: gbr = GradientBoostingRegressor(random_state=42, learning_rate=0.01, n_estimators=1000, subsample=0.8)
gbr.fit(x_train, y_train)
gbr_pred = gbr.predict(x_test)
mse_eval('GradientBoost Ensemble (lr=0.01, est=1000, subsample=0.8)', gbr_pred, y_test)
```

GradientBoost Ensemble (lr=0.01, est=1000, subsample=0.8)



	model	mse
0	ElasticNet(l1_ratio=0.8)	23.156236
1	Standard ElasticNet	22.694970
2	Ridge(alpha=1)	22.480476
3	Lasso(alpha=0.01)	22.210557
4	LinearRegression	22.098695
5	GradientBoost Ensemble (lr=0.01)	21.982151
6	Voting Ensemble	20.474757
7	Poly ElasticNet	15.710492
8	RandomForest Ensemble w/ Tuning	10.196040
9	RandomForest Ensemble	9.933307
10	GradientBoost Ensemble	8.956141
11	GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780
12	GradientBoost Ensemble (lr=0.01, est=1000)	8.546408



XGBoost

e**X**treme **G**radient **B**oosting

[공식 문서](#)

주요 특징

- scikit-learn 패키지가 아닙니다.
- 성능이 우수함
- GBM보다는 빠르고 성능도 향상되었습니다.
- 여전히 학습시간이 매우 느리다

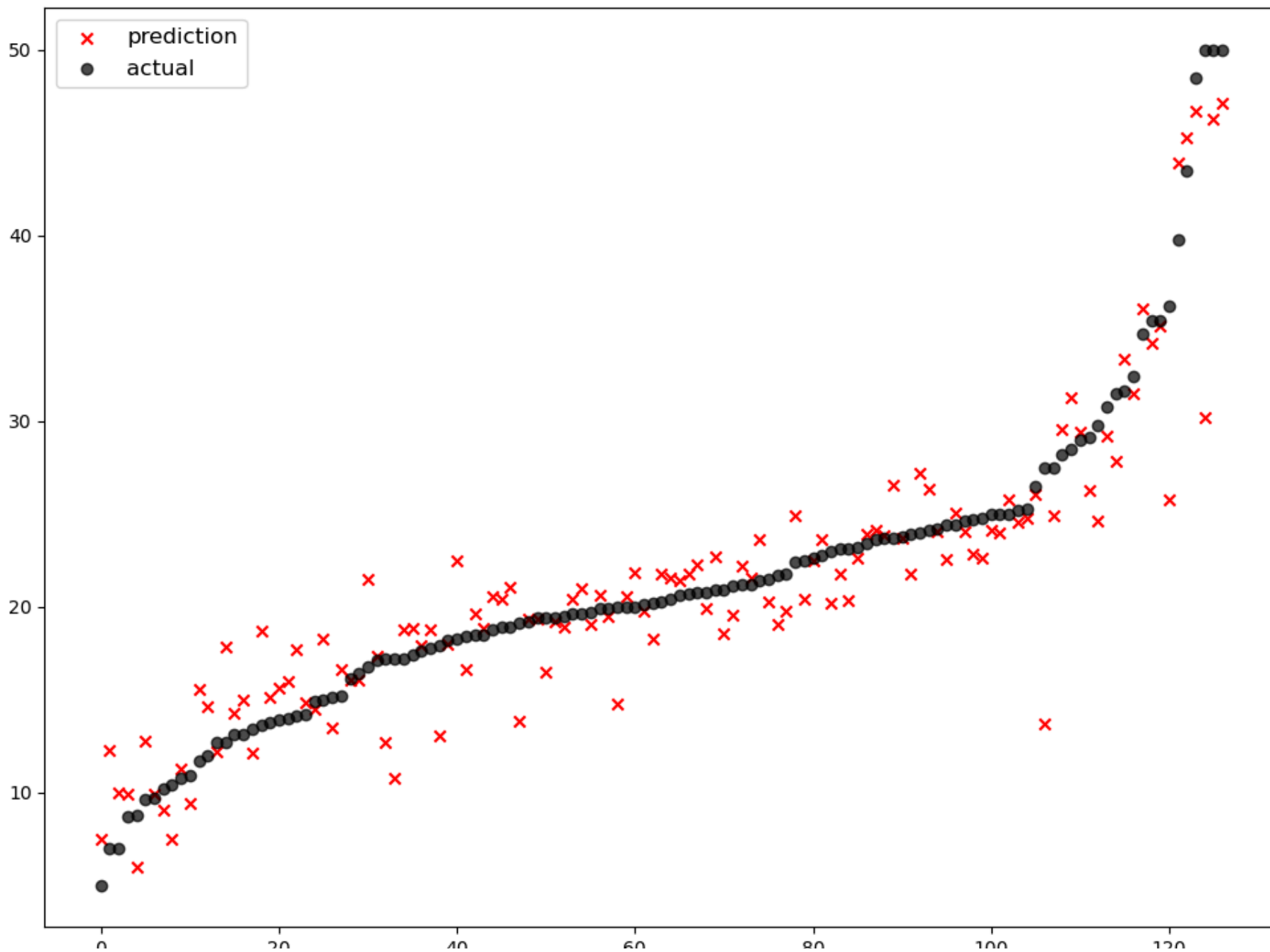
```
In [100... !pip install xgboost
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: xgboost in c:\users\user\appdata\roaming\python\python312\site-packages (2.1.4)
Requirement already satisfied: numpy in c:\programdata\anaconda3\lib\site-packages (from xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\programdata\anaconda3\lib\site-packages (from xgboost) (1.13.1)
```

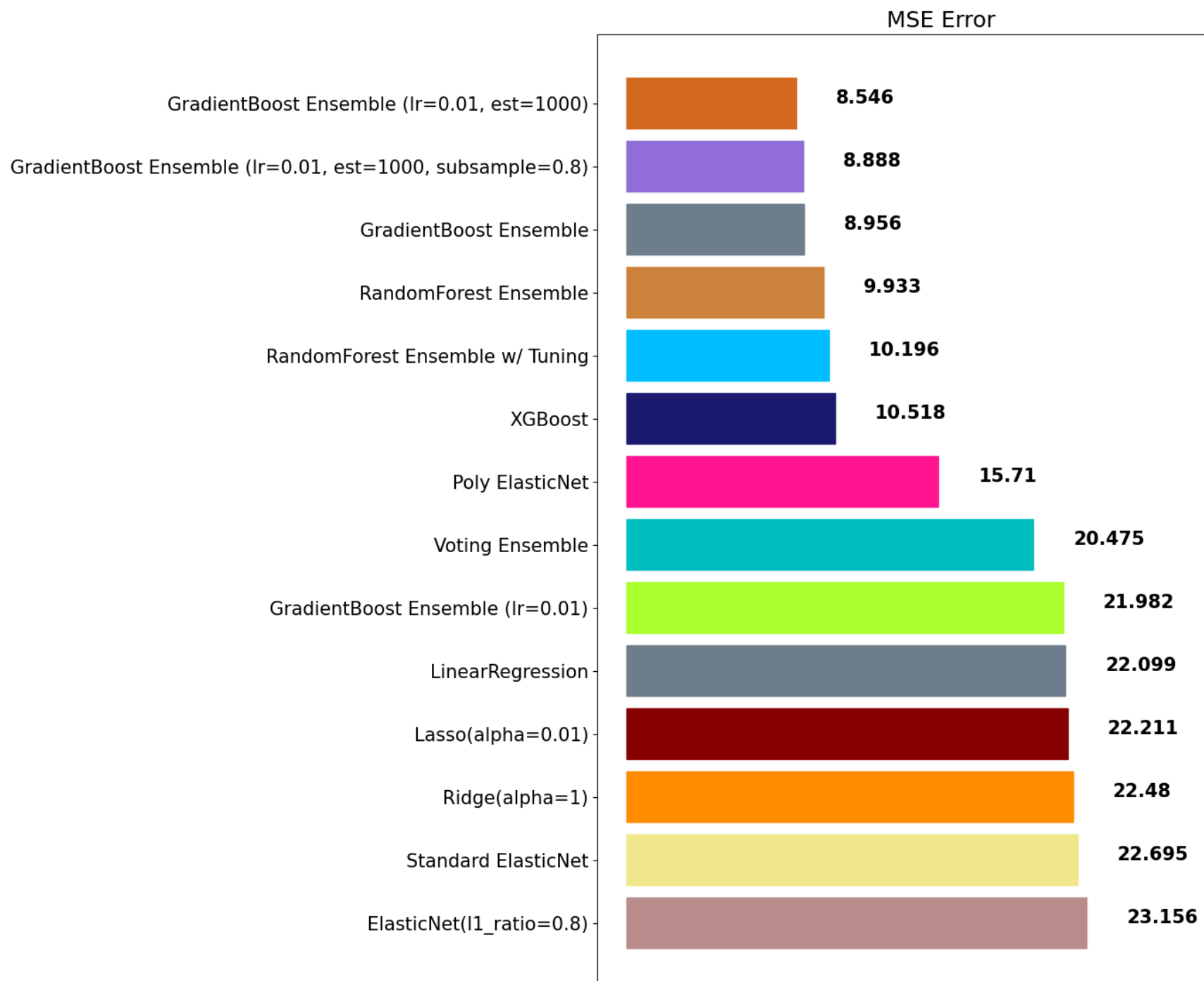
```
In [101... from xgboost import XGBRegressor, XGBClassifier
```

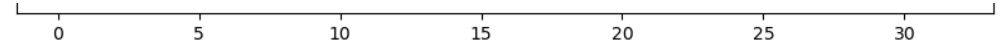
```
In [102... xgb = XGBRegressor(random_state=42)
xgb.fit(x_train, y_train)
xgb_pred = xgb.predict(x_test)
mse_eval('XGBoost', xgb_pred, y_test)
```


XGBoost



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			XGBoost	10.518450			
9			RandomForest Ensemble w/ Tuning	10.196040			
10			RandomForest Ensemble	9.933307			
11			GradientBoost Ensemble	8.956141			
12			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
13			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





주요 Hyperparameter

- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!
- n_jobs: CPU 사용 갯수
- learning_rate: 학습율. 너무 큰 학습율은 성능을 떨어뜨리고, 너무 작은 학습율은 학습이 느리다. 적절한 값을 찾아야함. n_estimators와 같이 튜닝. default=0.1
- n_estimators: 부스팅 스테이지 수. (랜덤포레스트 트리의 갯수 설정과 비슷한 개념). default=100
- max_depth: 트리의 깊이. 과대적합 방지용. default=3.
- subsample: 샘플 사용 비율. 과대적합 방지용. default=1.0
- max_features: 최대 사용할 feature의 비율. 과대적합 방지용. default=1.0

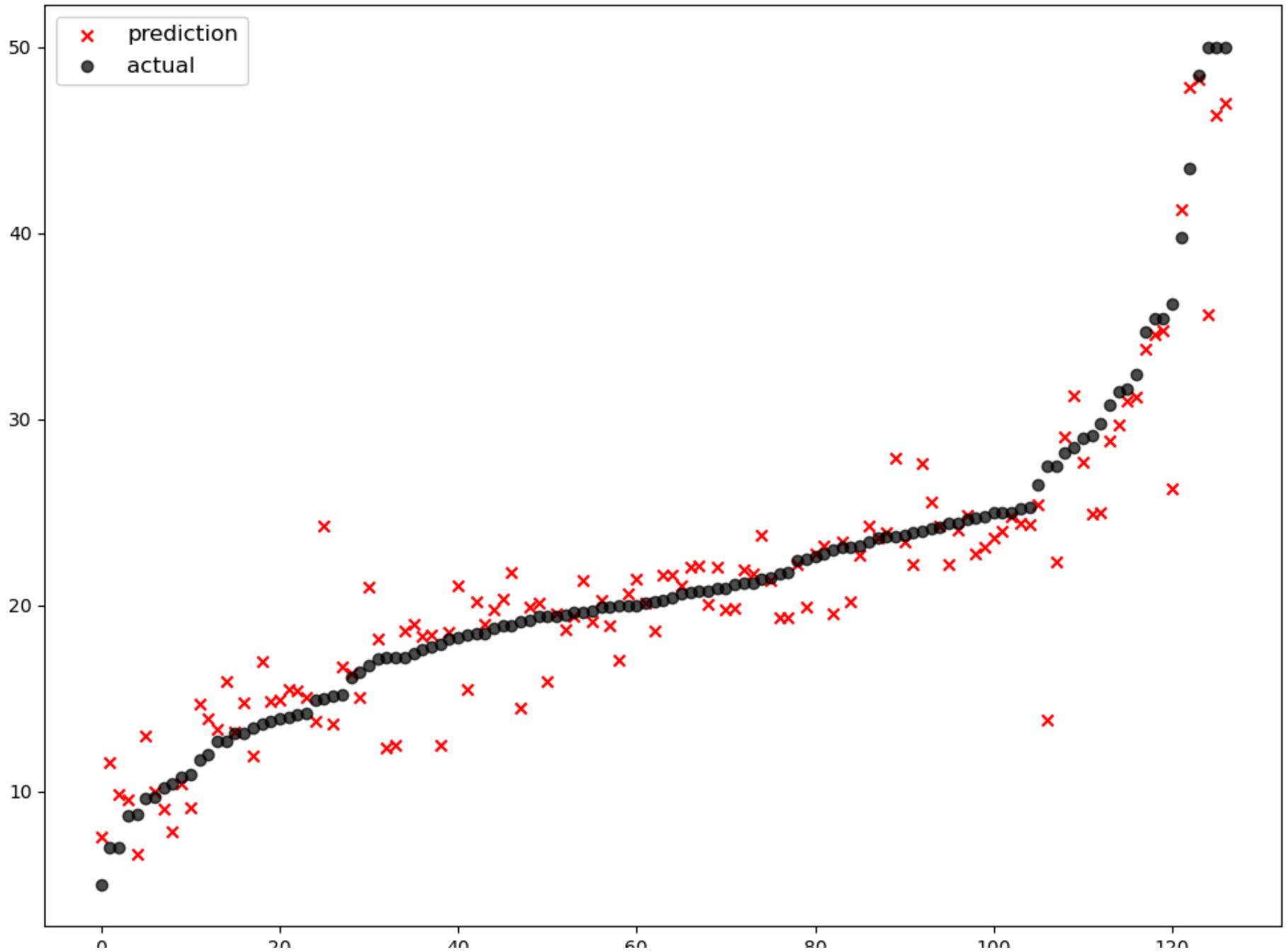
In [104...

```
xgb = XGBRegressor(random_state=42, learning_rate=0.01, n_estimators=1000, subsample=0.8, max_features=0.8, max_depth=7)
xgb.fit(x_train, y_train)
xgb_pred = xgb.predict(x_test)
mse_eval('XGBoost w/ Tuning', xgb_pred, y_test)
```

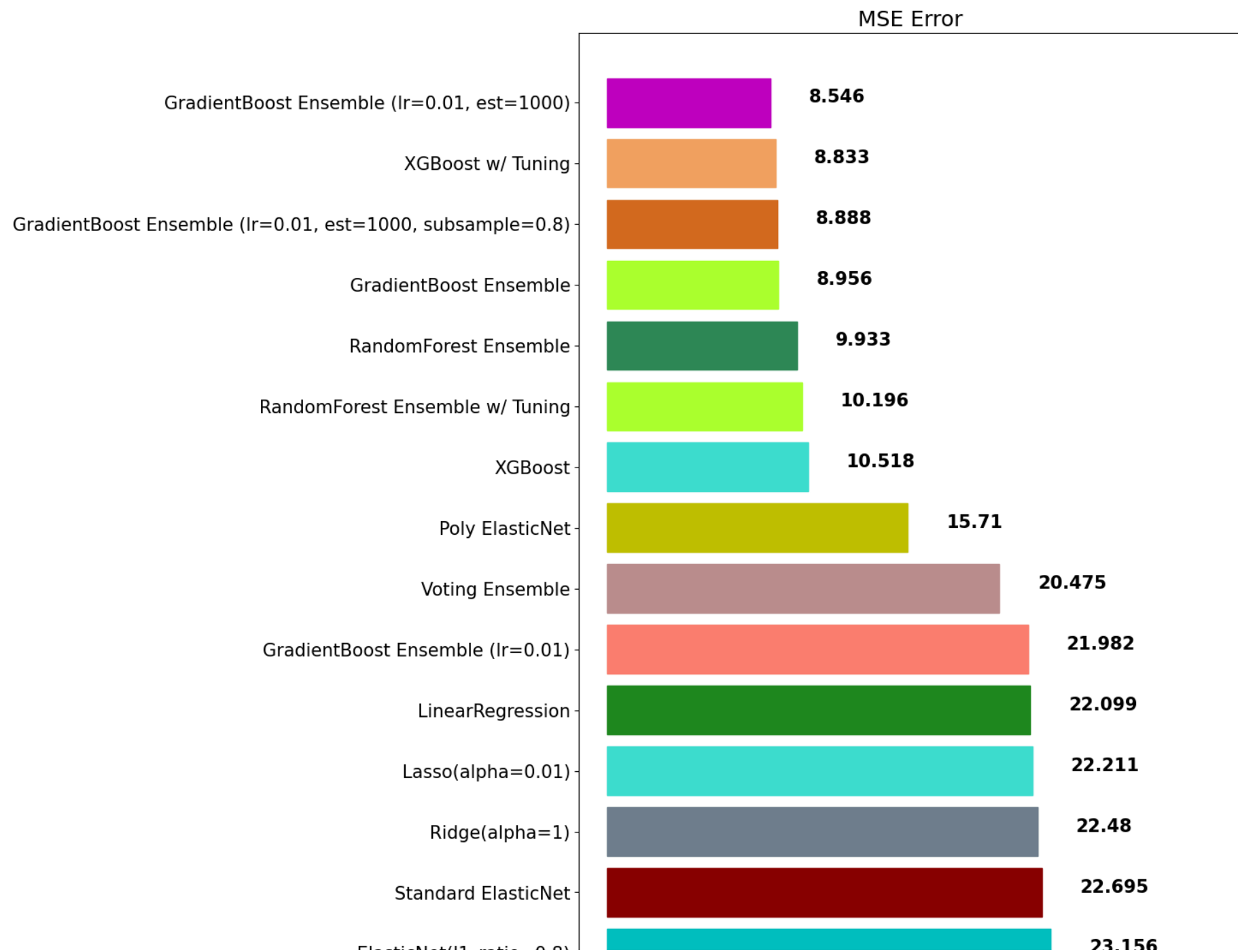
C:\Users\user\AppData\Roaming\Python\Python312\site-packages\xgboost\core.py:158: UserWarning: [10:57:26] WARNING: C:\buildkite-agent\builds\buildkite-windows-cpu-autoscaling-group-i-08cbc0333d8d4aae1-1\xgboost\xgboost-ci-windows\src\learner.cc:740: Parameters: { "max_features" } are not used.

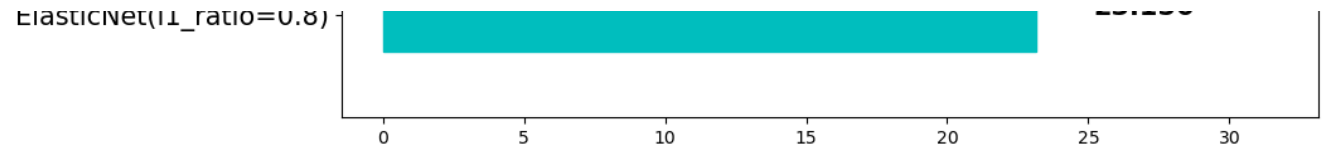
```
warnings.warn(smsg, UserWarning)
```

XGBoost w/ Tuning



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			XGBoost	10.518450			
9			RandomForest Ensemble w/ Tuning	10.196040			
10			RandomForest Ensemble	9.933307			
11			GradientBoost Ensemble	8.956141			
12			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
13			XGBoost w/ Tuning	8.832910			
14			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





LightGBM

[공식 문서](#)

주요 특징

- scikit-learn 패키지가 아닙니다.
- 성능이 우수함
- 속도도 매우 빠릅니다.

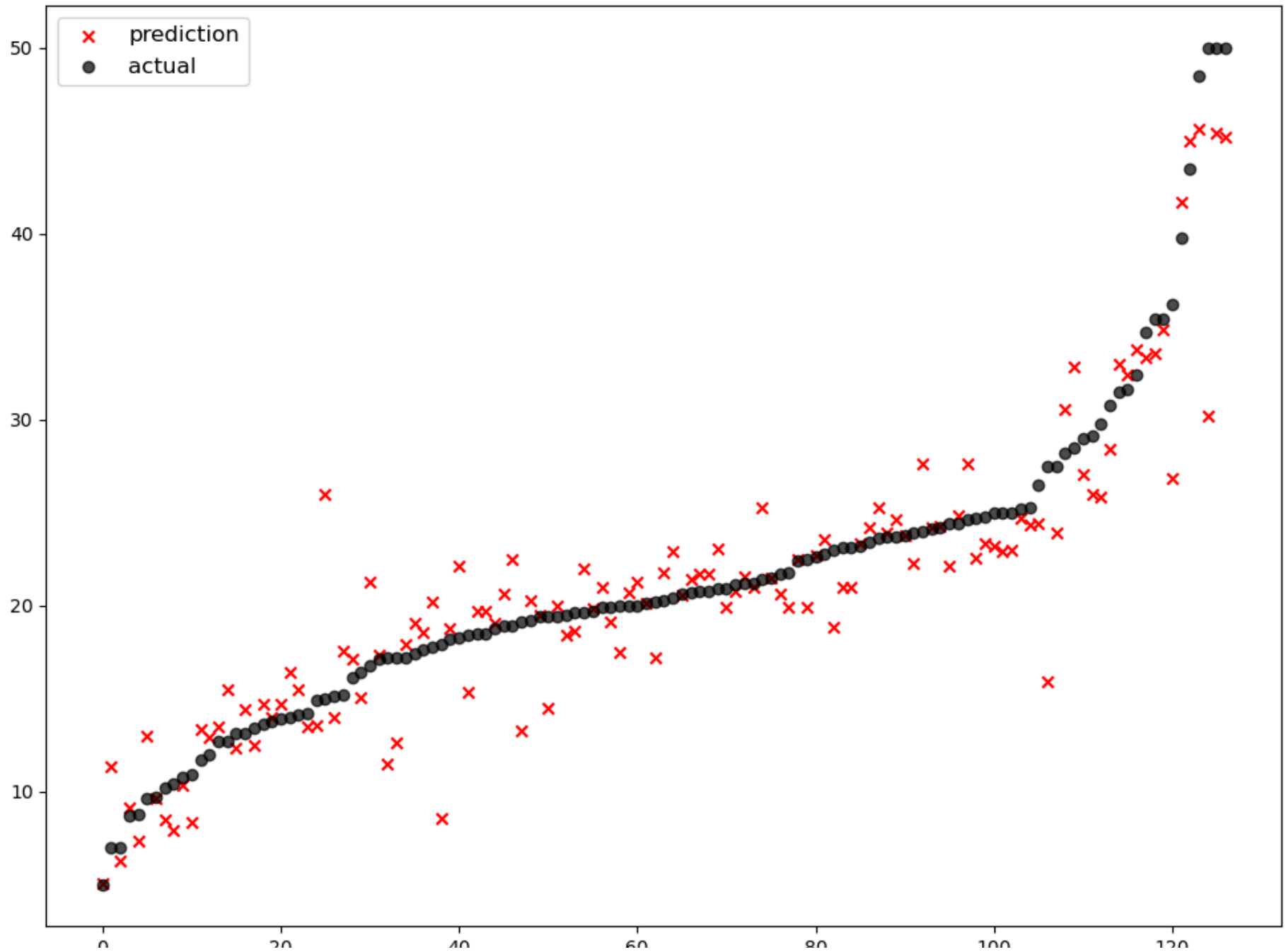
```
In [108... !pip install lightgbm
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: lightgbm in c:\users\user\appdata\roaming\python\python312\site-packages (4.6.0)
Requirement already satisfied: numpy>=1.17.0 in c:\programdata\anaconda3\lib\site-packages (from lightgbm) (1.26.4)
Requirement already satisfied: scipy in c:\programdata\anaconda3\lib\site-packages (from lightgbm) (1.13.1)
```

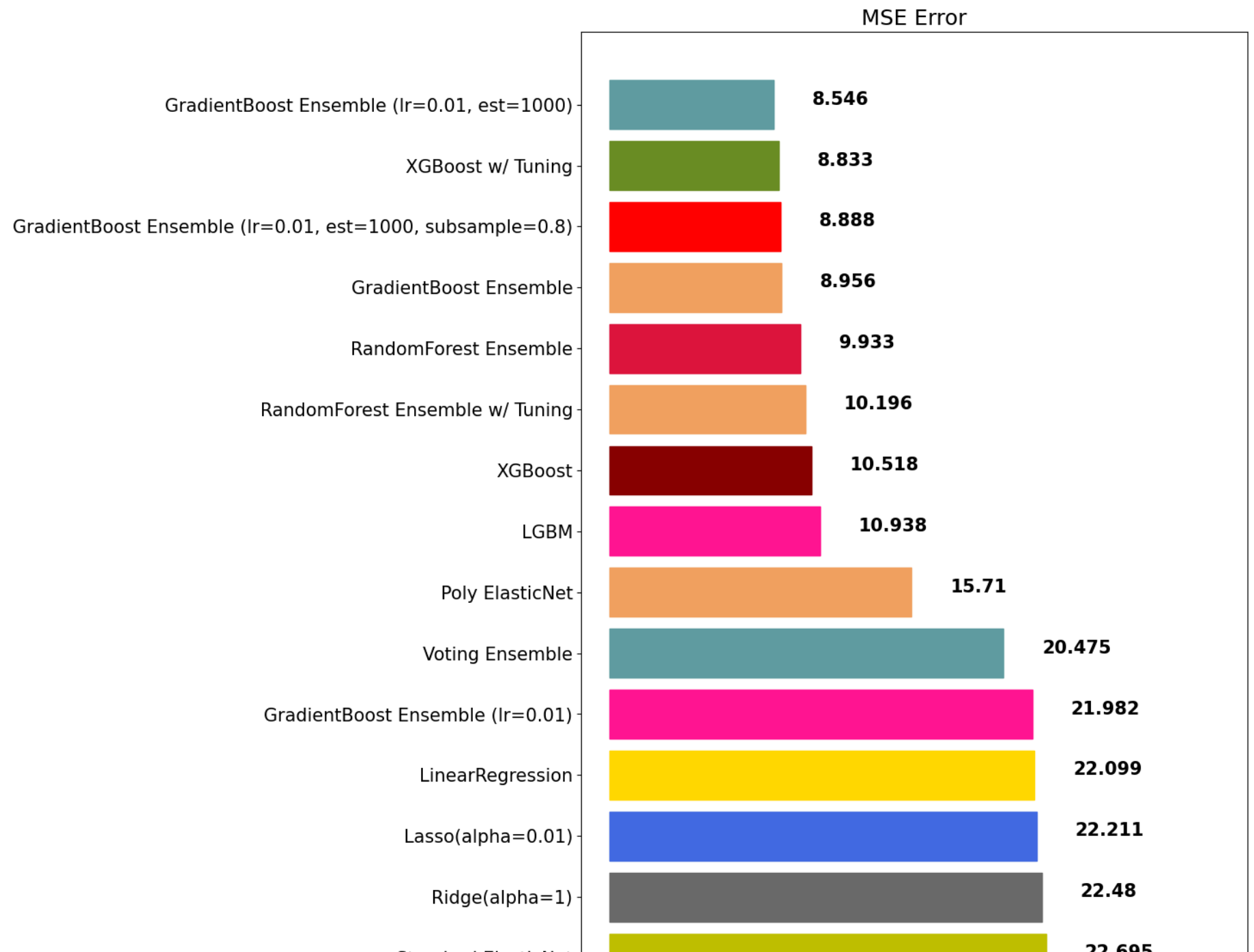
```
In [109... from lightgbm import LGBMRegressor, LGBMClassifier
```

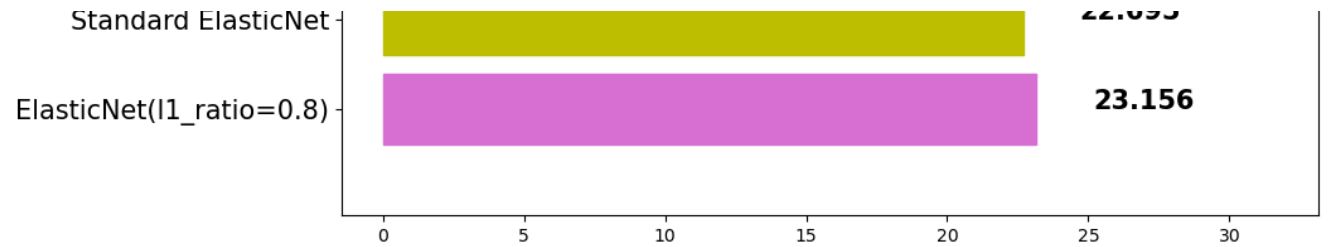
```
In [110... lgbm = LGBMRegressor(random_state=42, verbose=-1)
lgbm.fit(x_train, y_train)
lgbm_pred = lgbm.predict(x_test)
mse_eval('LGBM', lgbm_pred, y_test)
```


LGBM



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			LGBM	10.937726			
9			XGBoost	10.518450			
10			RandomForest Ensemble w/ Tuning	10.196040			
11			RandomForest Ensemble	9.933307			
12			GradientBoost Ensemble	8.956141			
13			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
14			XGBoost w/ Tuning	8.832910			
15			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			



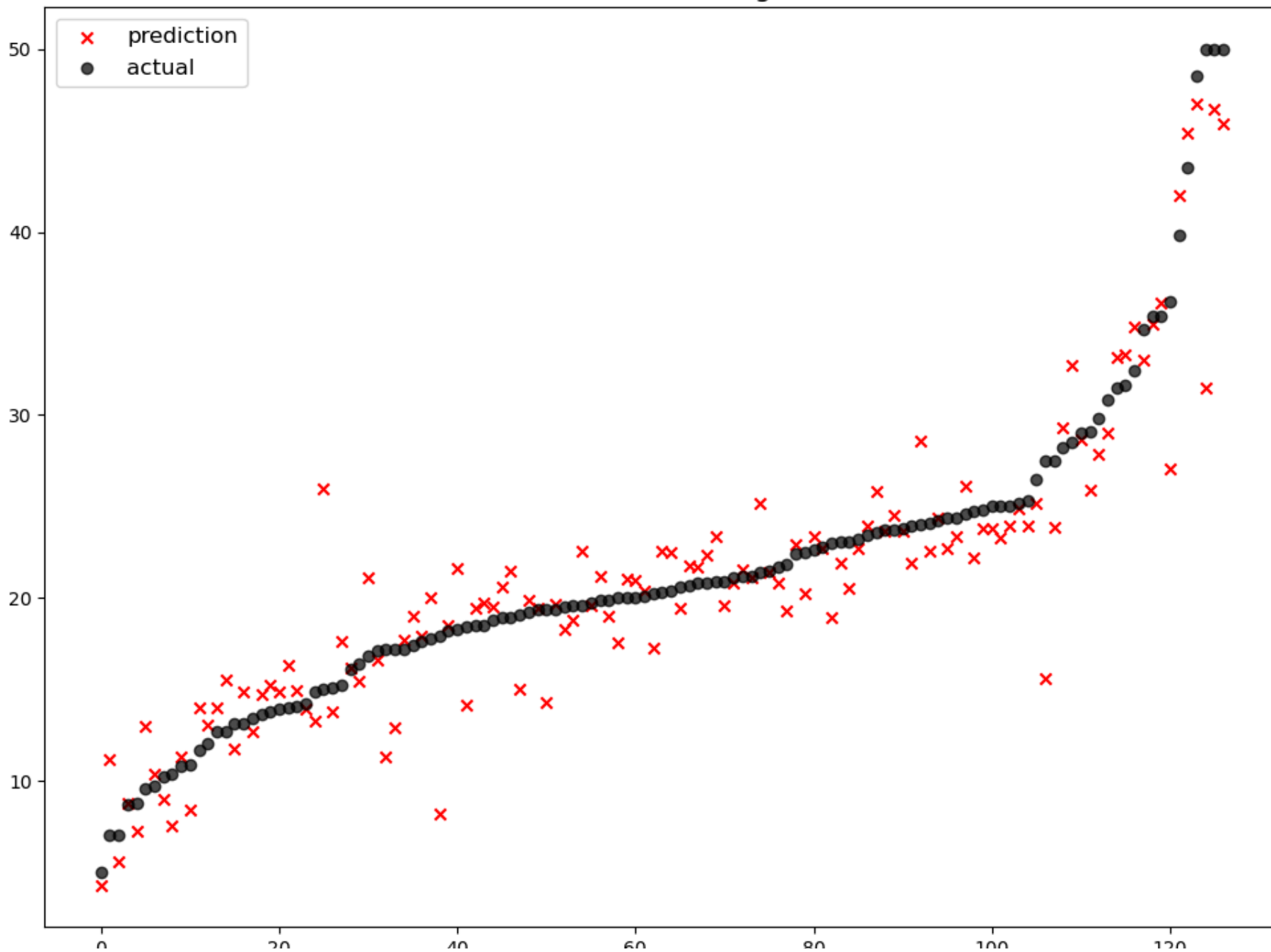


주요 Hyperparameter

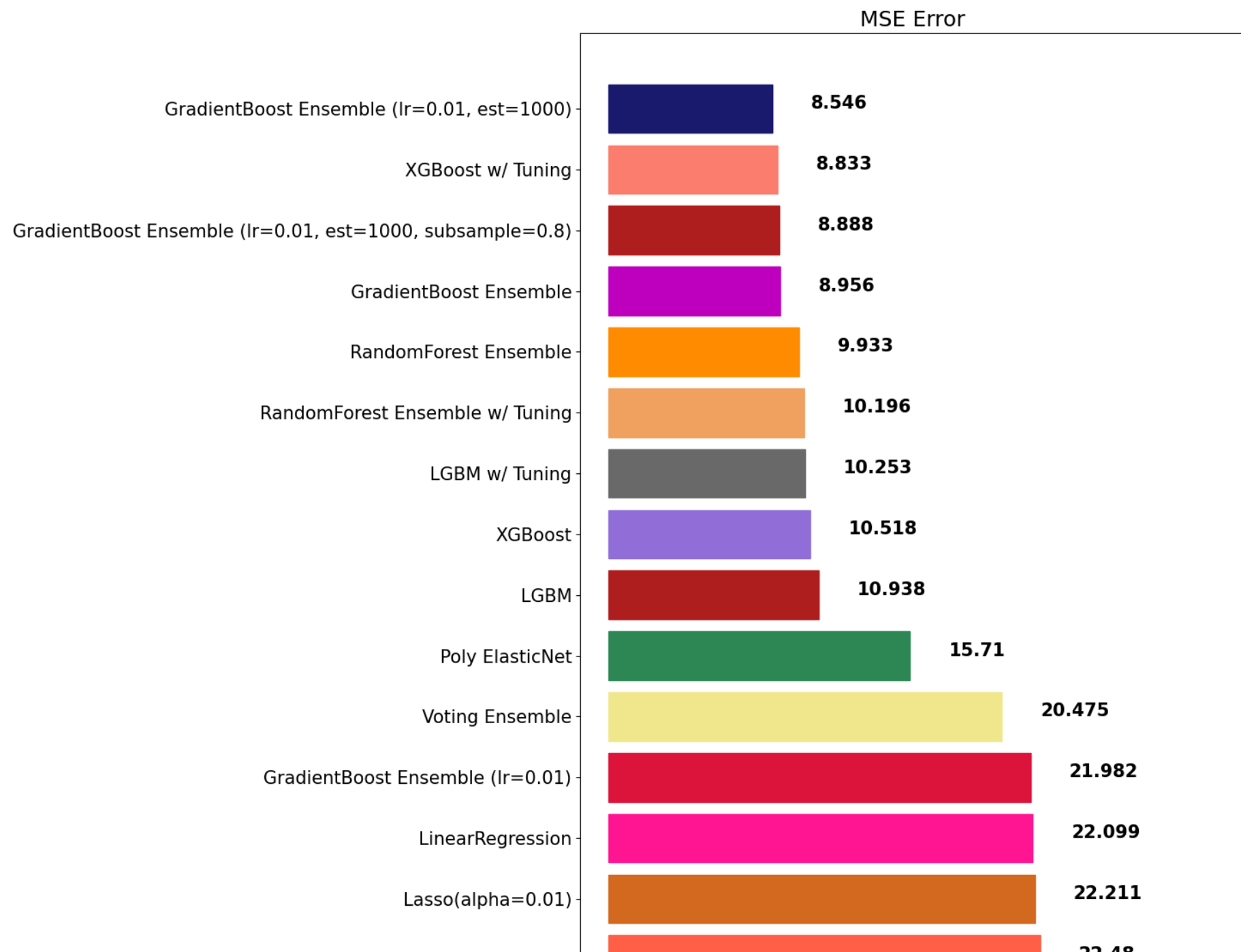
- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!
- n_jobs: CPU 사용 갯수
- learning_rate: 학습율. 너무 큰 학습율은 성능을 떨어뜨리고, 너무 작은 학습율은 학습이 느리다. 적절한 값을 찾아야함. n_estimators와 같이 튜닝. default=0.1
- n_estimators: 부스팅 스테이지 수. (랜덤포레스트 트리의 갯수 설정과 비슷한 개념). default=100
- max_depth: 트리의 깊이. 과대적합 방지용. default=3.
- colsample_bytree: 샘플 사용 비율 (max_features와 비슷한 개념). 과대적합 방지용. default=1.0

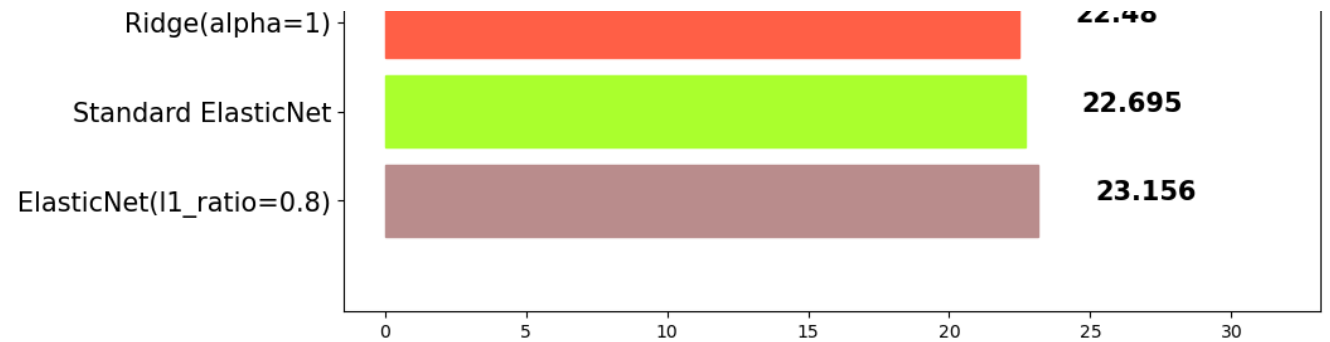
```
In [112... lgbm = LGBMRegressor(random_state=42, learning_rate=0.01, n_estimators=2000, colsample_bytree=0.9, subsample=0.9, max_depth=7)
lgbm.fit(x_train, y_train)
lgbm_pred = lgbm.predict(x_test)
mse_eval('LGBM w/ Tuning', lgbm_pred, y_test)
```

LGBM w/ Tuning



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			LGBM	10.937726			
9			XGBoost	10.518450			
10			LGBM w/ Tuning	10.252945			
11			RandomForest Ensemble w/ Tuning	10.196040			
12			RandomForest Ensemble	9.933307			
13			GradientBoost Ensemble	8.956141			
14			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
15			XGBoost w/ Tuning	8.832910			
16			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





Stacking

개별 모델이 예측한 데이터를 기반으로 **final_estimator** 종합하여 예측을 수행합니다.

- 성능을 극으로 끌어올릴 때 활용하기도 합니다.
- 과대적합을 유발할 수 있습니다. (특히, 데이터셋이 적은 경우)

공식문서

```
In [117... from sklearn.ensemble import StackingRegressor
```

```
In [118... stack_models = [  
    ('elasticnet', poly_pipeline),  
    ('randomforest', rfr),  
    ('gbr', gbr),  
    ('lgbm', lgbm),  
]
```

```
In [119... stack_reg = StackingRegressor(stack_models, final_estimator=xgb, n_jobs=-1)
```

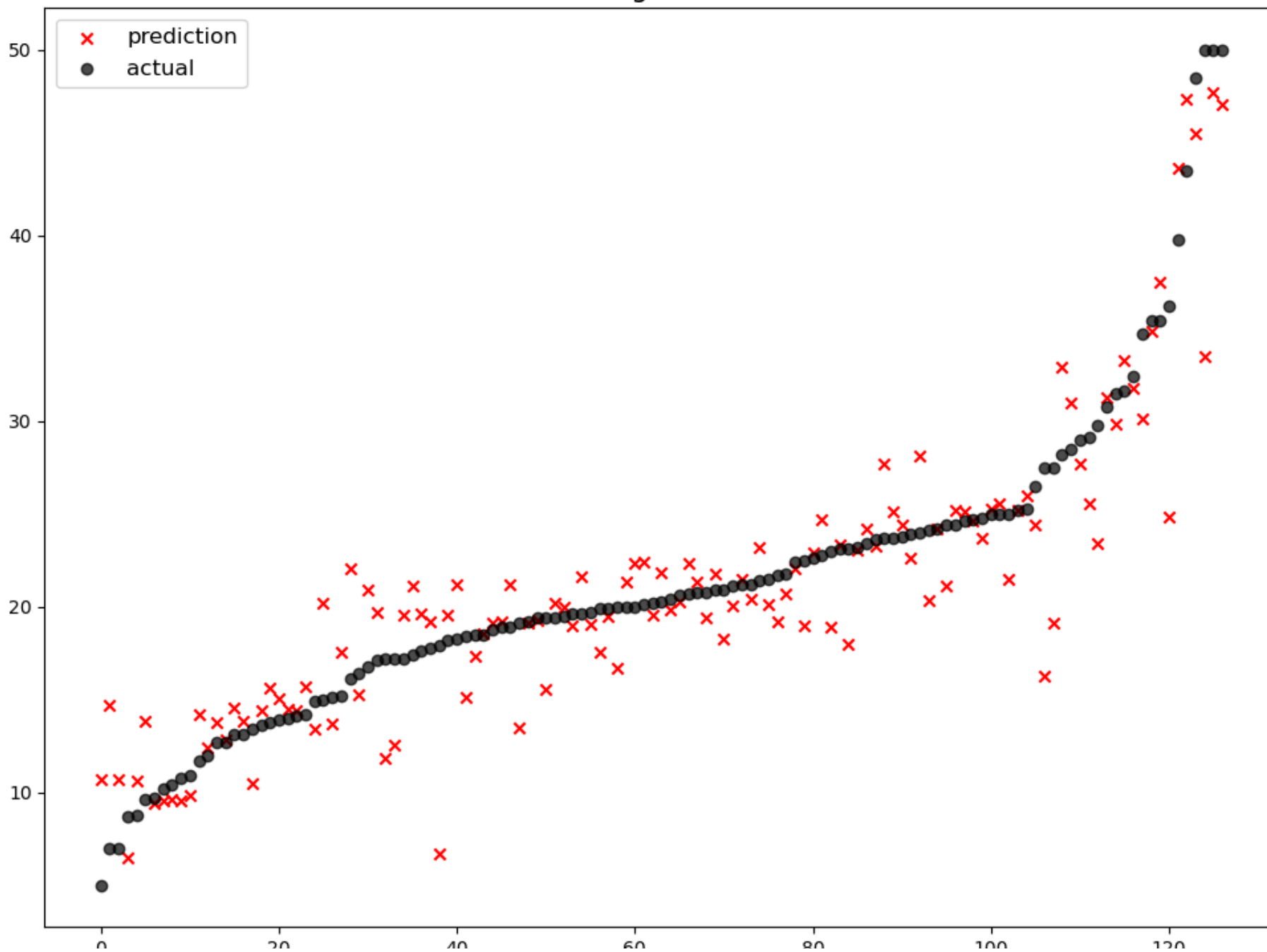
```
In [120... stack_reg.fit(x_train, y_train)  
stack_pred = stack_reg.predict(x_test)  
mse_eval('Stacking Ensemble', stack_pred, y_test)
```



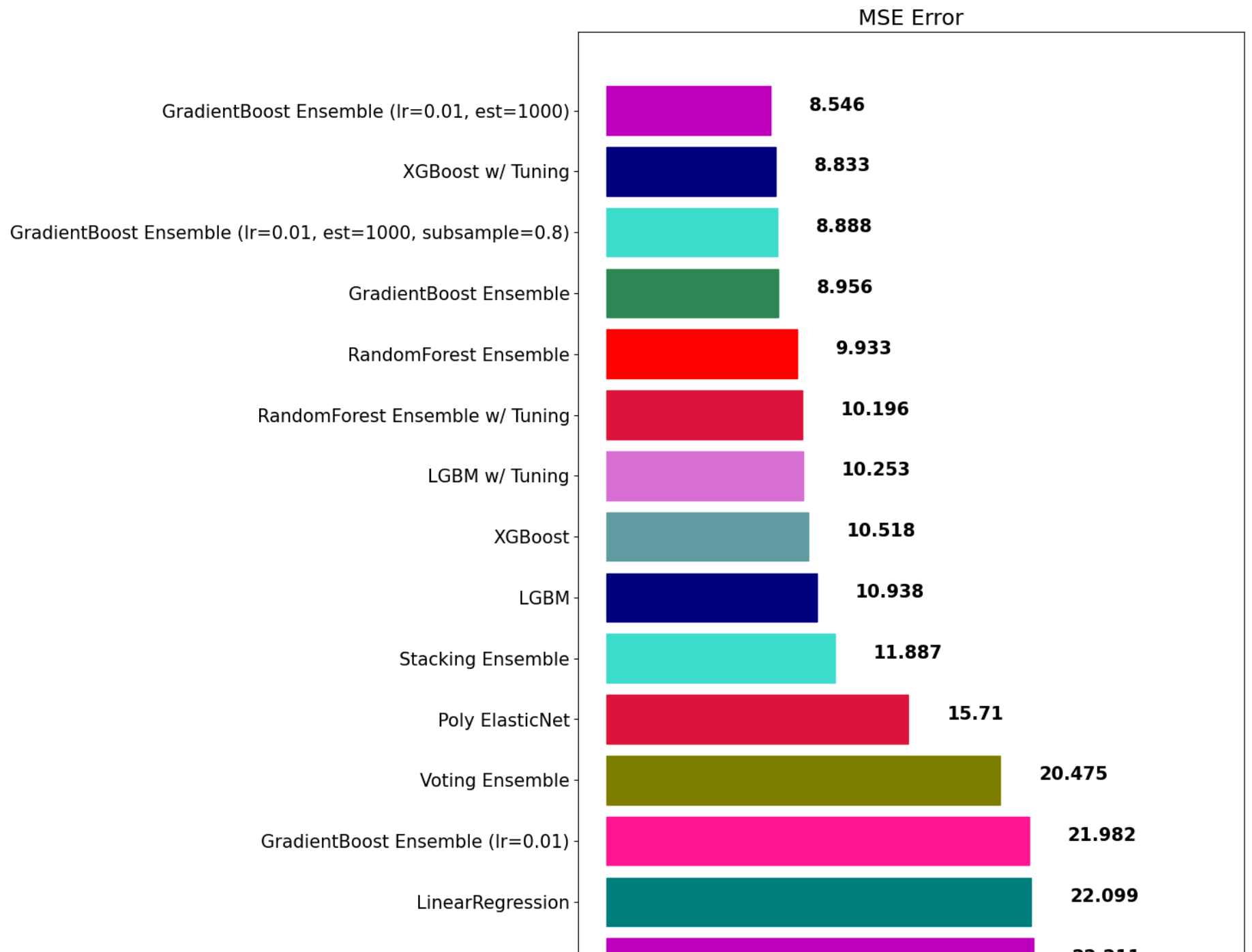
```
C:\Users\user\AppData\Roaming\Python\Python312\site-packages\xgboost\core.py:158: UserWarning: [10:58:18] WARNING: C:\buildkite-agent\builds\buildkite-windows-cpu-autoscaling-group-i-08cbc0333d8d4aae1-1\xgboost\xgboost-ci-windows\src\learner.cc:740: Parameters: { "max_features" } are not used.
```

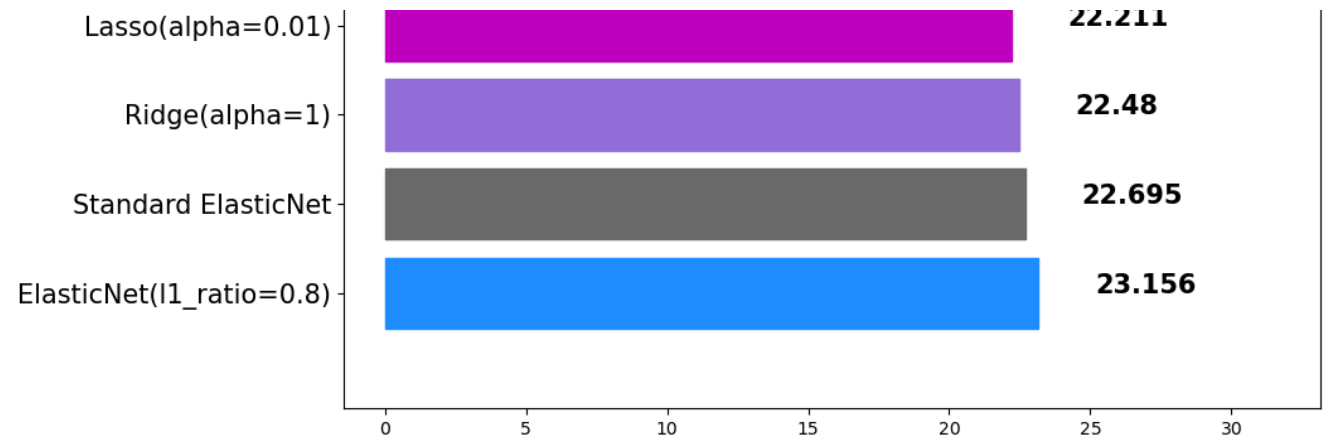
```
warnings.warn(smsg, UserWarning)
```

Stacking Ensemble



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			Stacking Ensemble	11.886880			
9			LGBM	10.937726			
10			XGBoost	10.518450			
11			LGBM w/ Tuning	10.252945			
12			RandomForest Ensemble w/ Tuning	10.196040			
13			RandomForest Ensemble	9.933307			
14			GradientBoost Ensemble	8.956141			
15			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
16			XGBoost w/ Tuning	8.832910			
17			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





Weighted Blending

각 모델의 예측값에 대하여 weight를 곱하여 최종 output 계산

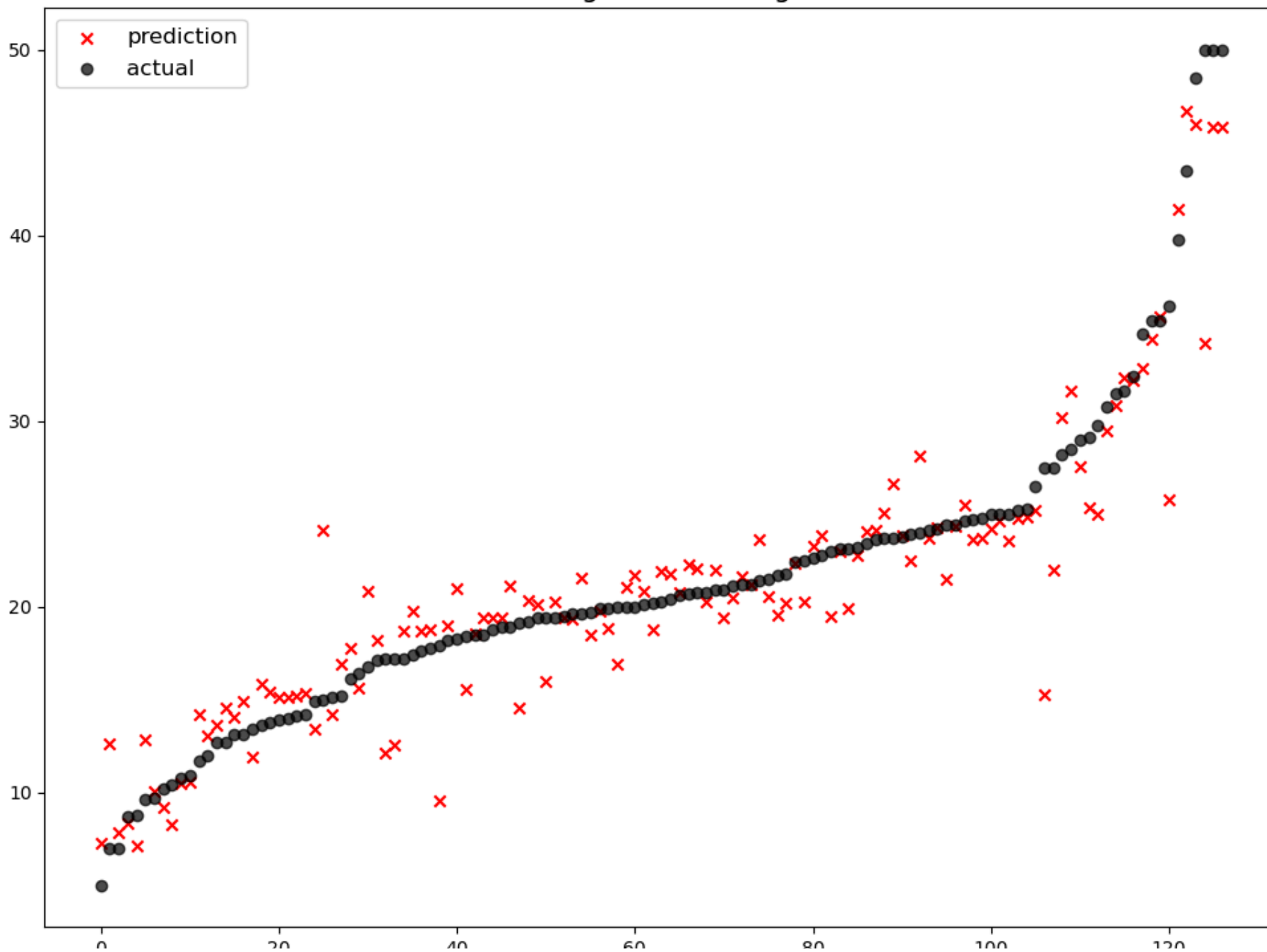
- 모델에 대한 가중치를 조절하여, 최종 output을 산출합니다.
- **가중치의 합은 1.0**이 되도록 합니다.

```
In [123... final_outputs = {\n    'elasticnet': poly_pred,\n    'randomforest': rfr_pred,\n    'gbr': gbr_pred,\n    'xgb': xgb_pred,\n    'lgbm': lgbm_pred,\n    'stacking': stack_pred,\n}
```

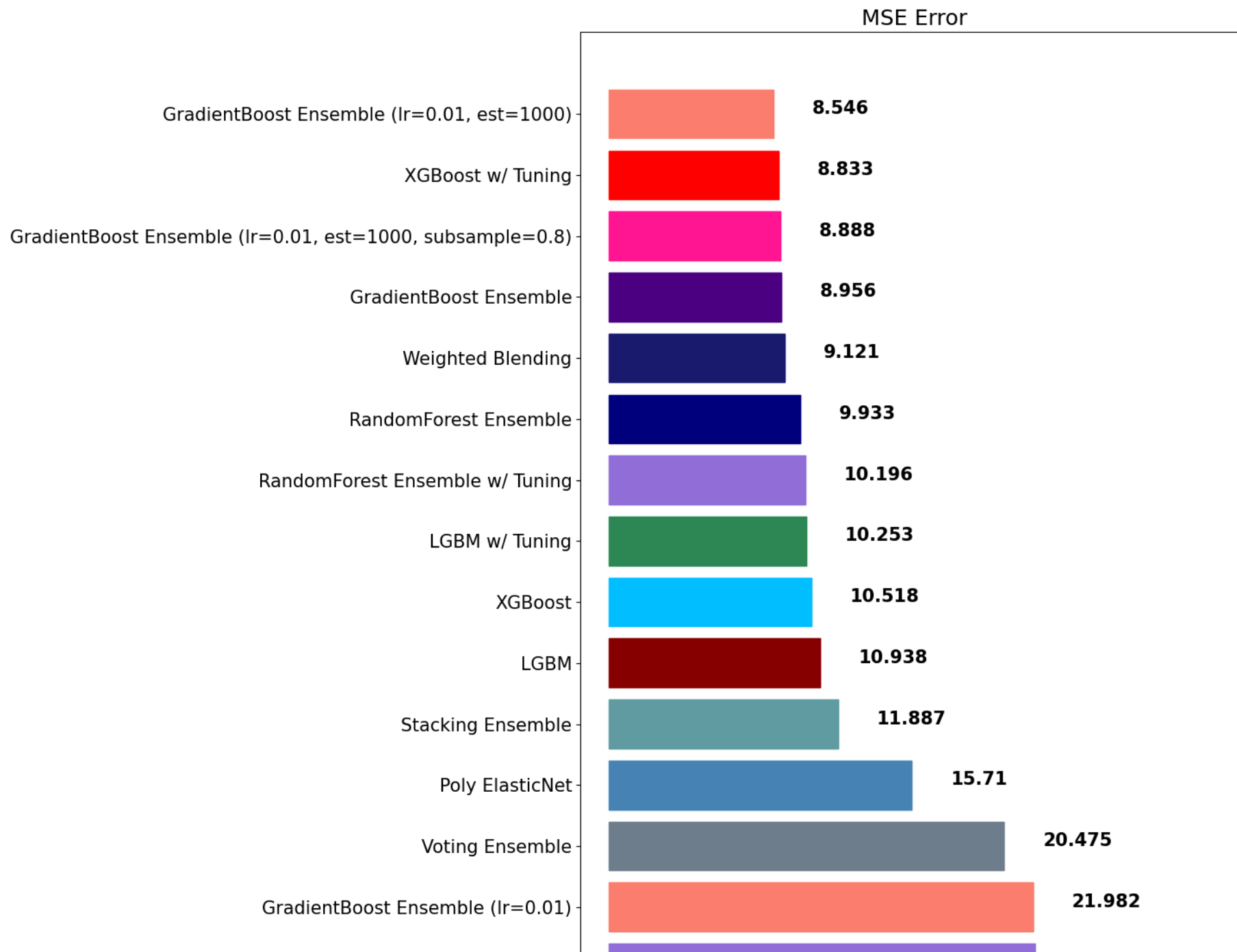
```
In [124... final_prediction=\nfinal_outputs['elasticnet'] * 0.1\
```

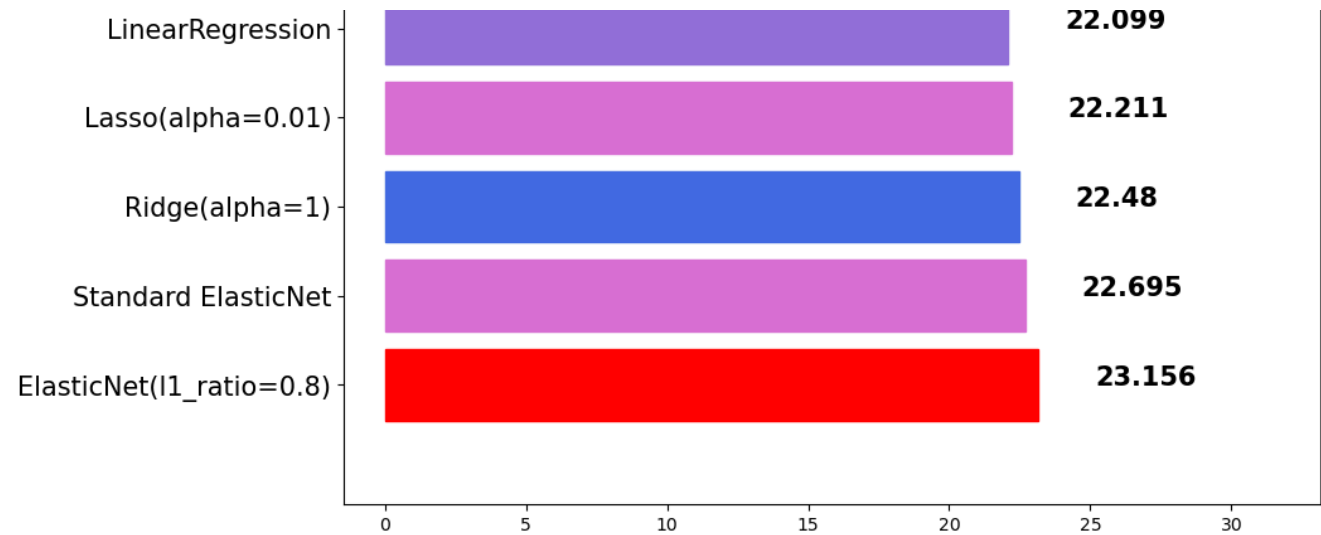
```
In [125... mse_eval('Weighted Blending', final_prediction, y_test)
```

Weighted Blending



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			Stacking Ensemble	11.886880			
9			LGBM	10.937726			
10			XGBoost	10.518450			
11			LGBM w/ Tuning	10.252945			
12			RandomForest Ensemble w/ Tuning	10.196040			
13			RandomForest Ensemble	9.933307			
14			Weighted Blending	9.121321			
15			GradientBoost Ensemble	8.956141			
16			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
17			XGBoost w/ Tuning	8.832910			
18			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





앙상블 모델을 정리하며

1. 앙상블은 대체적으로 단일 모델 대비 성능이 좋습니다.
2. 앙상블을 앙상블하는 기법인 Stacking과 Weighted Blending도 참고해 볼만 합니다.
3. 앙상블 모델은 적절한 **Hyperparameter 튜닝**이 중요합니다.
4. 앙상블 모델은 대체적으로 학습시간이 더 오래 걸립니다.
5. 따라서, 모델 튜닝을 하는 데에 걸리는 시간이 오래 소요됩니다.

In []:

Cross Validation

- Cross Validation이란 모델을 평가하는 하나의 방법입니다.
- K-겹 교차검증(K-fold Cross Validation)을 많이 활용합니다.

K-겹 교차검증

- K-겹 교차 검증은 모든 데이터가 최소 한 번은 테스트셋으로 쓰이도록 합니다. 아래의 그림을 보면, 데이터를 5개로 쪼개 매번 테스트셋을 바꿔나가는 것을 볼 수 있습니다.

[예시]

- Estimation 1일때,

학습데이터: [B, C, D, E] / 검증데이터: [A]

- Estimation 2일때,

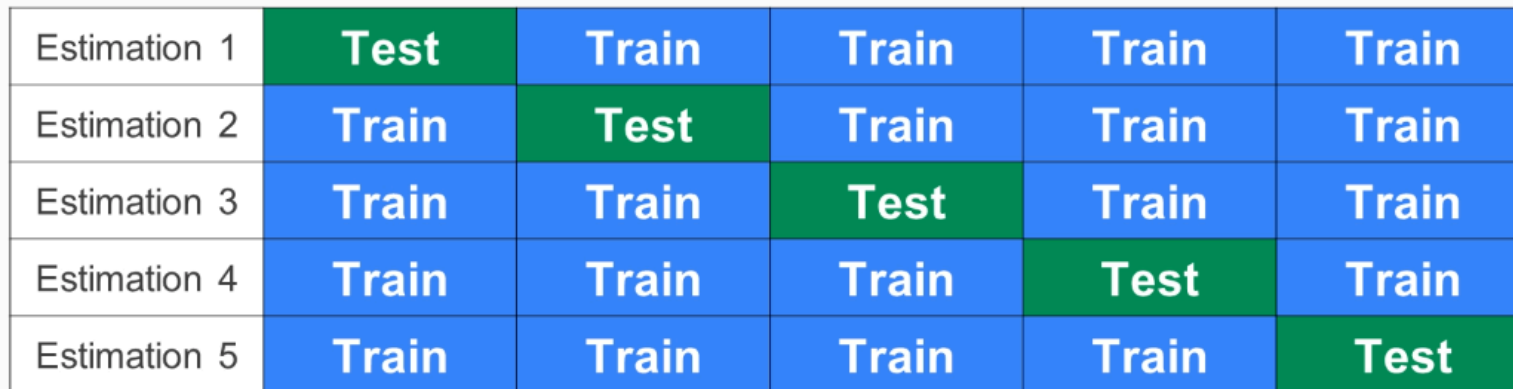
학습데이터: [A, C, D, E] / 검증데이터: [B]

In [130... `Image('b04c27c5-7e3f-428a-9aa6-bb3ebcd3584c.png', width=800)`

Out[130...

5-fold CV

DATASET



Estimation 1	Test	Train	Train	Train	Train
Estimation 2	Train	Test	Train	Train	Train
Estimation 3	Train	Train	Test	Train	Train
Estimation 4	Train	Train	Train	Test	Train
Estimation 5	Train	Train	Train	Train	Test

K-Fold Cross Validation Set 만들기

In [132... `from sklearn.model_selection import KFold`

```
In [133... n_splits = 5
kfold = KFold(n_splits=n_splits, random_state=42, shuffle=True)
```

```
In [134... df.head()
```

```
Out[134...      CRIM  ZN  INDUS  CHAS  NOX   RM  AGE   DIS  RAD  TAX  PTRATIO    B  LSTAT  MEDV
0  0.00632  18.0   2.31    0  0.538  6.575  65.2  4.0900   1  296.0    15.3  396.90   4.98   24.0
1  0.02731   0.0   7.07    0  0.469  6.421  78.9  4.9671   2  242.0    17.8  396.90   9.14   21.6
2  0.02729   0.0   7.07    0  0.469  7.185  61.1  4.9671   2  242.0    17.8  392.83   4.03   34.7
3  0.03237   0.0   2.18    0  0.458  6.998  45.8  6.0622   3  222.0    18.7  394.63   2.94   33.4
4  0.06905   0.0   2.18    0  0.458  7.147  54.2  6.0622   3  222.0    18.7  396.90   5.33   36.2
```

```
In [135... X = np.array(df.drop('MEDV', axis=1))
Y = np.array(df['MEDV'])
```

```
In [136... lgbm_fold = LGBMRegressor(random_state=42)
```

```
In [137... i = 1
total_error = 0
for train_index, test_index in kfold.split(X):
    x_train_fold, x_test_fold = X[train_index], X[test_index]
    y_train_fold, y_test_fold = Y[train_index], Y[test_index]
    lgbm_pred_fold = lgbm_fold.fit(x_train_fold, y_train_fold).predict(x_test_fold)
    error = mean_squared_error(lgbm_pred_fold, y_test_fold)
    print('Fold = {}, prediction score = {:.2f}'.format(i, error))
    total_error += error
    i+=1
print('---'*10)
print('Average Error: %s' % (total_error / n_splits))
```

```
Fold = 1, prediction score = 8.34
Fold = 2, prediction score = 10.40
Fold = 3, prediction score = 17.58
Fold = 4, prediction score = 6.94
Fold = 5, prediction score = 12.16
```

```
-----
Average Error: 11.083201392666322
```

In []:

Hyperparameter 튜닝

- hyperparameter 튜닝시 경우의 수가 너무 많습니다.
- 따라서, 우리는 자동화할 필요가 있습니다.

sklearn 패키지에서 자주 사용되는 hyperparameter 튜닝을 돕는 클래스는 다음 2가지가 있습니다.

1. **RandomizedSearchCV**
2. **GridSearchCV**

적용하는 방법

1. 사용할 Search 방법을 선택합니다.
2. hyperparameter 도메인을 설정합니다. (`max_depth` , `n_estimators` ..등등)
3. 학습을 시킨 후, 기다립니다.
4. 도출된 결과 값을 모델에 적용하고 성능을 비교합니다.

RandomizedSearchCV

- 모든 매개 변수 값이 시도되는 것이 아니라 지정된 분포에서 고정 된 수의 매개 변수 설정이 샘플링됩니다.
- 시도 된 매개 변수 설정의 수는 `n_iter` 에 의해 제공됩니다.

주요 Hyperparameter (LGBM)

- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!
- n_jobs: CPU 사용 갯수
- learning_rate: 학습율. 너무 큰 학습율은 성능을 떨어뜨리고, 너무 작은 학습율은 학습이 느리다. 적절한 값을 찾아야함. n_estimators와 같이 튜닝. default=0.1
- n_estimators: 부스팅 스테이지 수. (랜덤포레스트 트리의 갯수 설정과 비슷한 개념). default=100
- max_depth: 트리의 깊이. 과대적합 방지용. default=3.
- colsample_bytree: 샘플 사용 비율 (max_features와 비슷한 개념). 과대적합 방지용. default=1.0

```
In [145... params = {  
    'n_estimators': [200, 500, 1000, 2000],  
    'learning_rate': [0.1, 0.05, 0.01],  
    'max_depth': [6, 7, 8],  
    'colsample_bytree': [0.8, 0.9, 1.0],  
    'subsample': [0.8, 0.9, 1.0],  
}
```

```
In [146... from sklearn.model_selection import RandomizedSearchCV
```

n_iter 값을 조절하여, 총 몇 회의 시도를 진행할 것인지 정의합니다.

(회수가 늘어나면, 더 좋은 parameter를 찾을 확률은 올라가지만, 그만큼 시간이 오래걸립니다.)

```
In [148... clf = RandomizedSearchCV(LGBMRegressor(), params, random_state=42, cv=3, n_iter=25, scoring='neg_mean_squared_error')
```

```
In [149... clf.fit(x_train, y_train)
```

```
Out[149... RandomizedSearchCV ⓘ ⓘ  
  best_estimator_: LGBMRegressor  
    ▶ LGBMRegressor
```

In [150... `clf.best_score_`

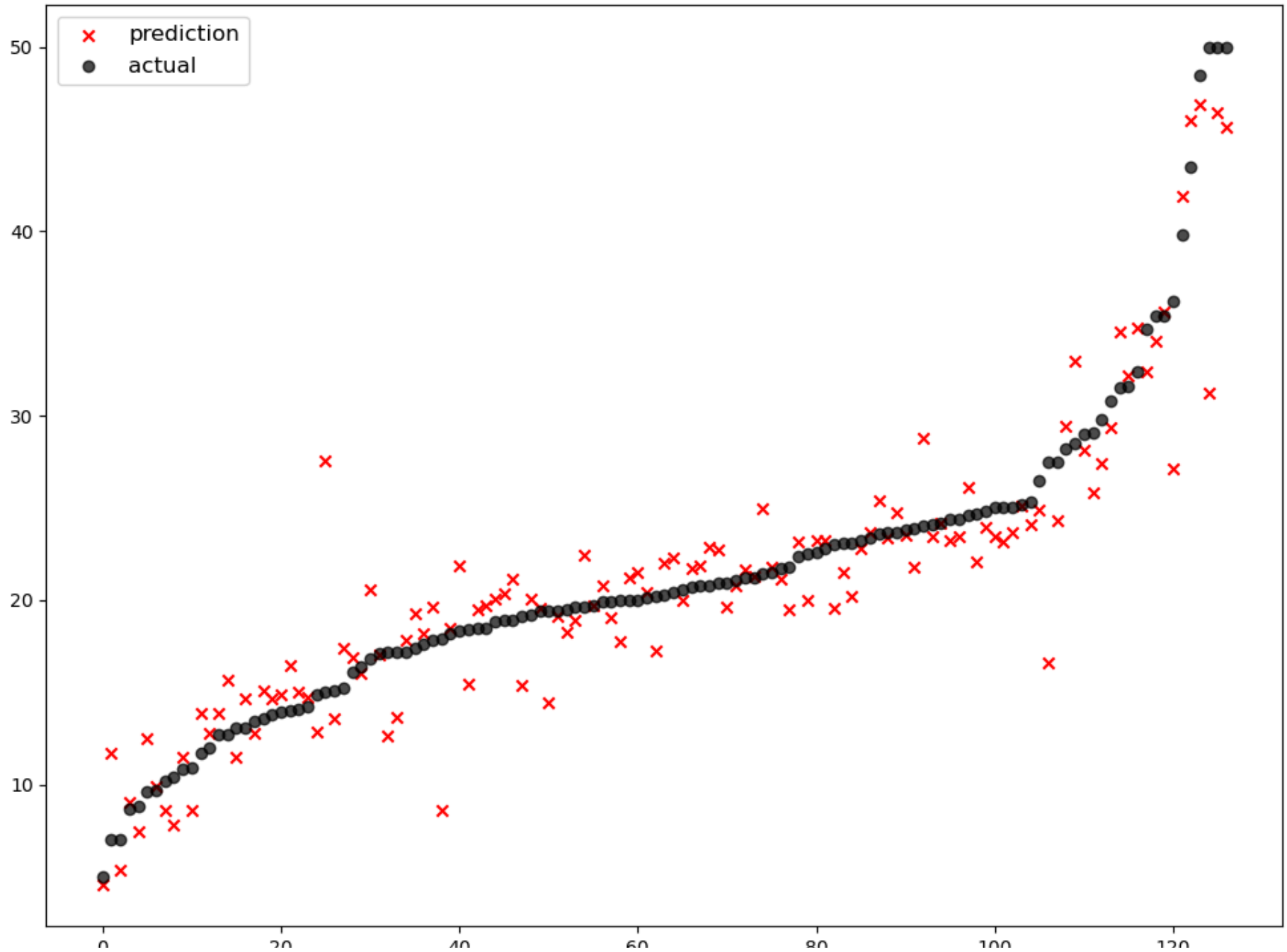
Out[150... `-13.707228623244996`

In [151... `clf.best_params_`

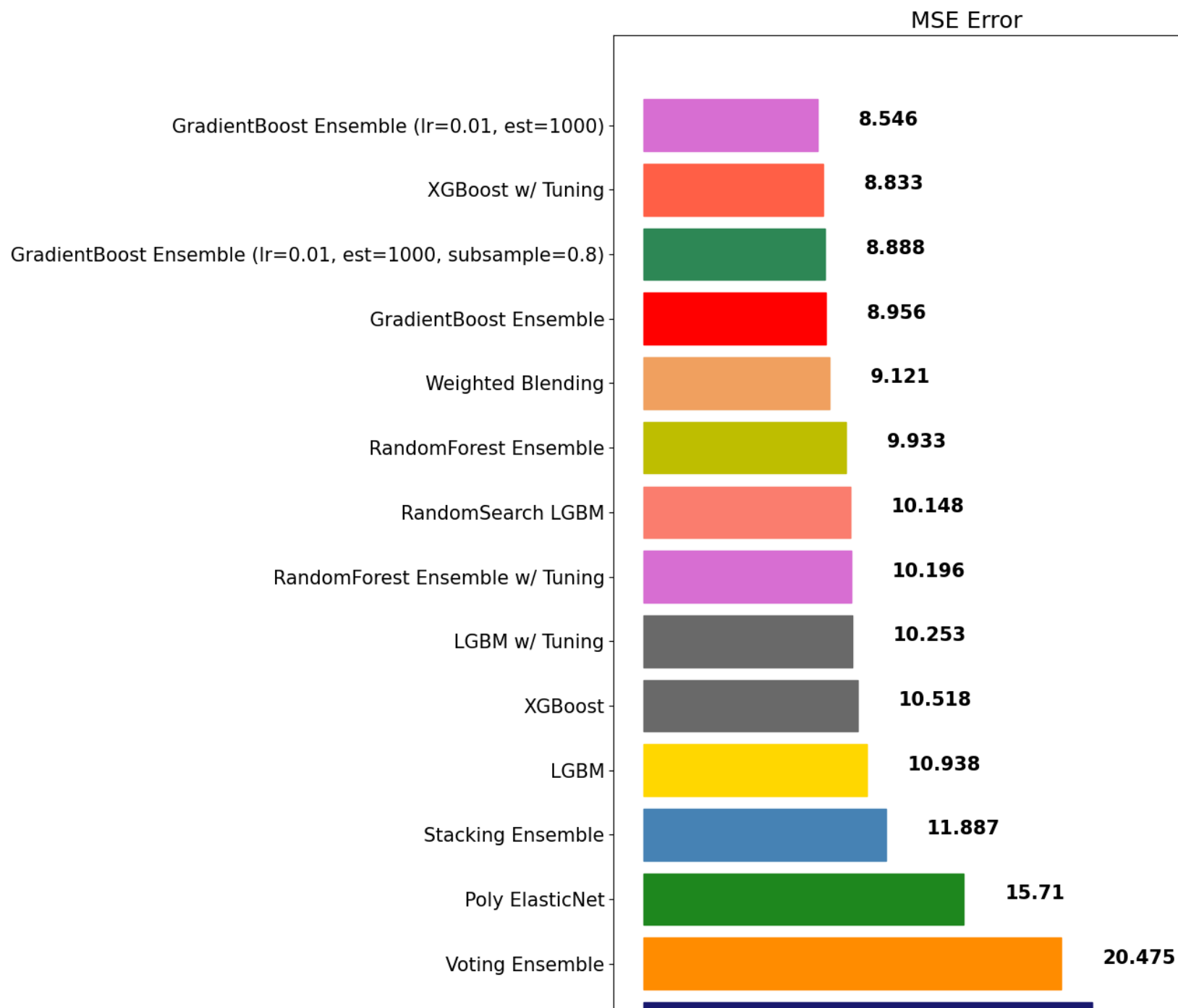
Out[151... `{'subsample': 0.9,
 'n_estimators': 2000,
 'max_depth': 6,
 'learning_rate': 0.01,
 'colsample_bytree': 0.8}`

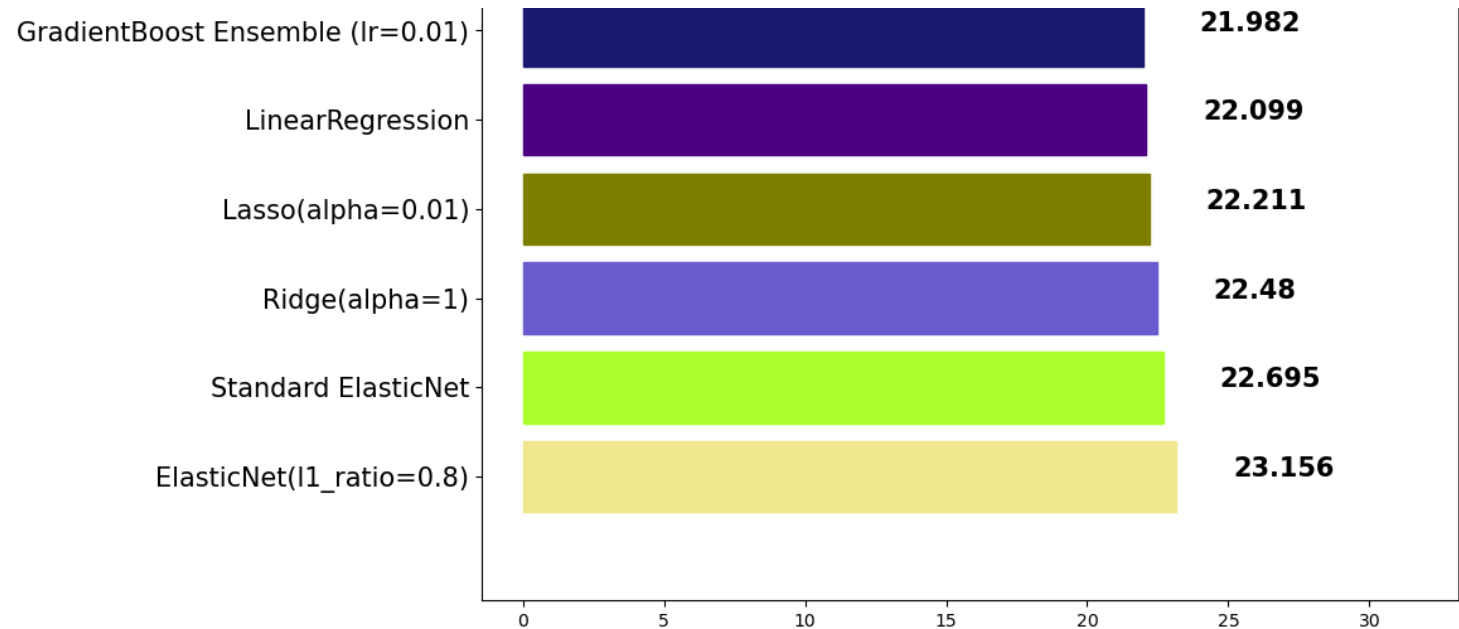
In [152... `lgbm_best = LGBMRegressor(n_estimators=2000, subsample=0.8, max_depth=7, learning_rate=0.01, colsample_bytree=0.8)
lgbm_best_pred = lgbm_best.fit(x_train, y_train).predict(x_test)
mse_eval('RandomSearch LGBM', lgbm_best_pred, y_test)`

RandomSearch LGBM



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			Stacking Ensemble	11.886880			
9			LGBM	10.937726			
10			XGBoost	10.518450			
11			LGBM w/ Tuning	10.252945			
12			RandomForest Ensemble w/ Tuning	10.196040			
13			RandomSearch LGBM	10.147690			
14			RandomForest Ensemble	9.933307			
15			Weighted Blending	9.121321			
16			GradientBoost Ensemble	8.956141			
17			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
18			XGBoost w/ Tuning	8.832910			
19			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





GridSearchCV

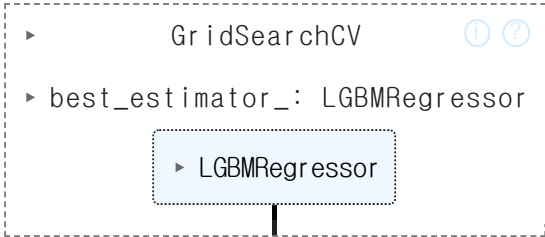
- 모든 매개 변수 값에 대하여 **완전 탐색**을 시도합니다.
- 따라서, 최적화할 parameter가 많다면, **시간이 매우 오래**걸립니다.

```
In [155... params = {  
    'n_estimators': [500, 1000],  
    'learning_rate': [0.1, 0.05, 0.01],  
    'max_depth': [7, 8],  
    'colsample_bytree': [0.8, 0.9],  
    'subsample': [0.8, 0.9],  
}
```

```
In [156... from sklearn.model_selection import GridSearchCV
```

```
In [157... grid_search = GridSearchCV(LGBMRegressor(), params, cv=3, n_jobs=-1, scoring='neg_mean_squared_error')
```

In [158... `grid_search.fit(x_train, y_train)`

Out[158... 

In [159... `grid_search.best_score_`

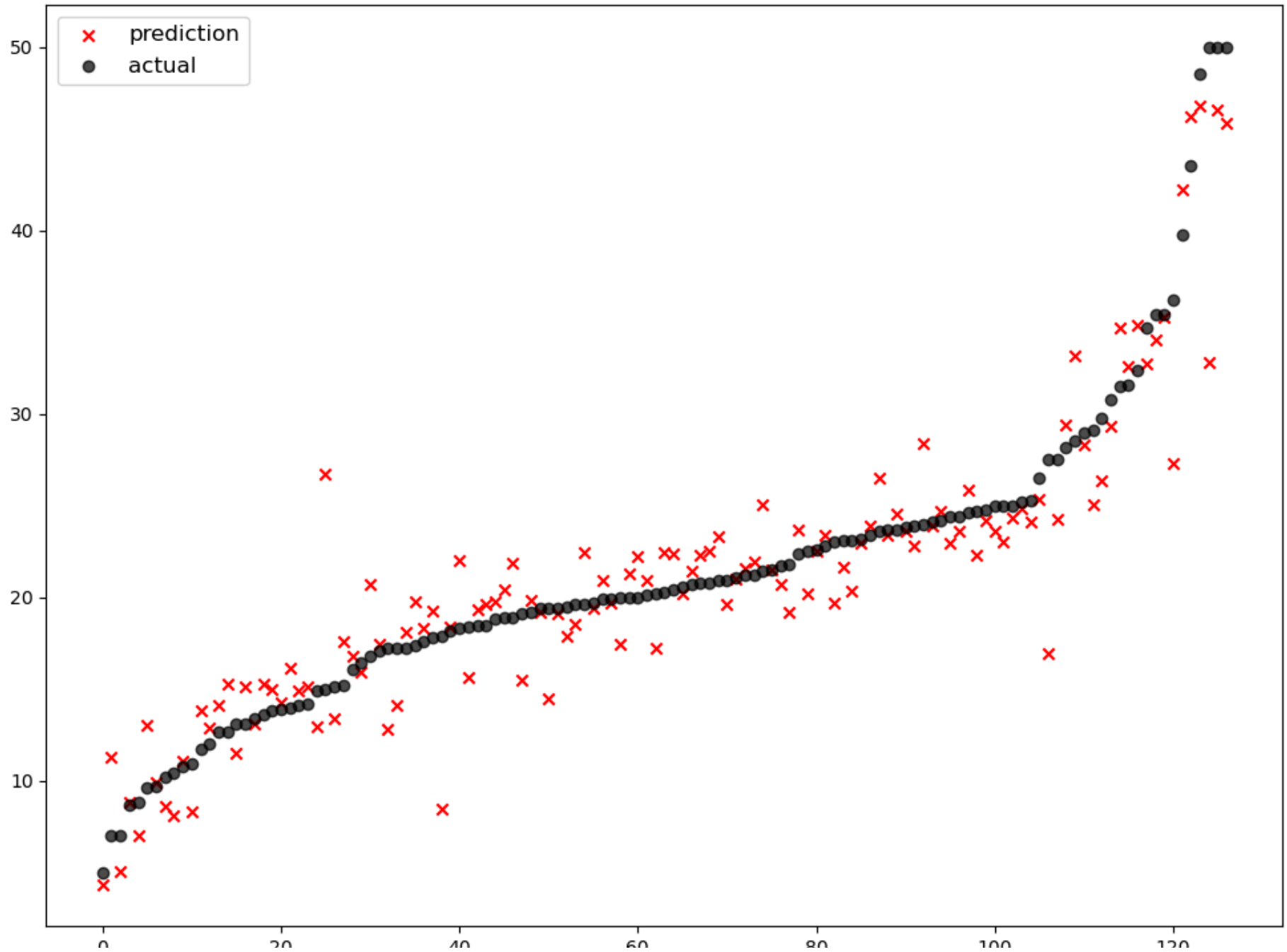
Out[159... -13.598939419010335

In [160... `grid_search.best_params_`

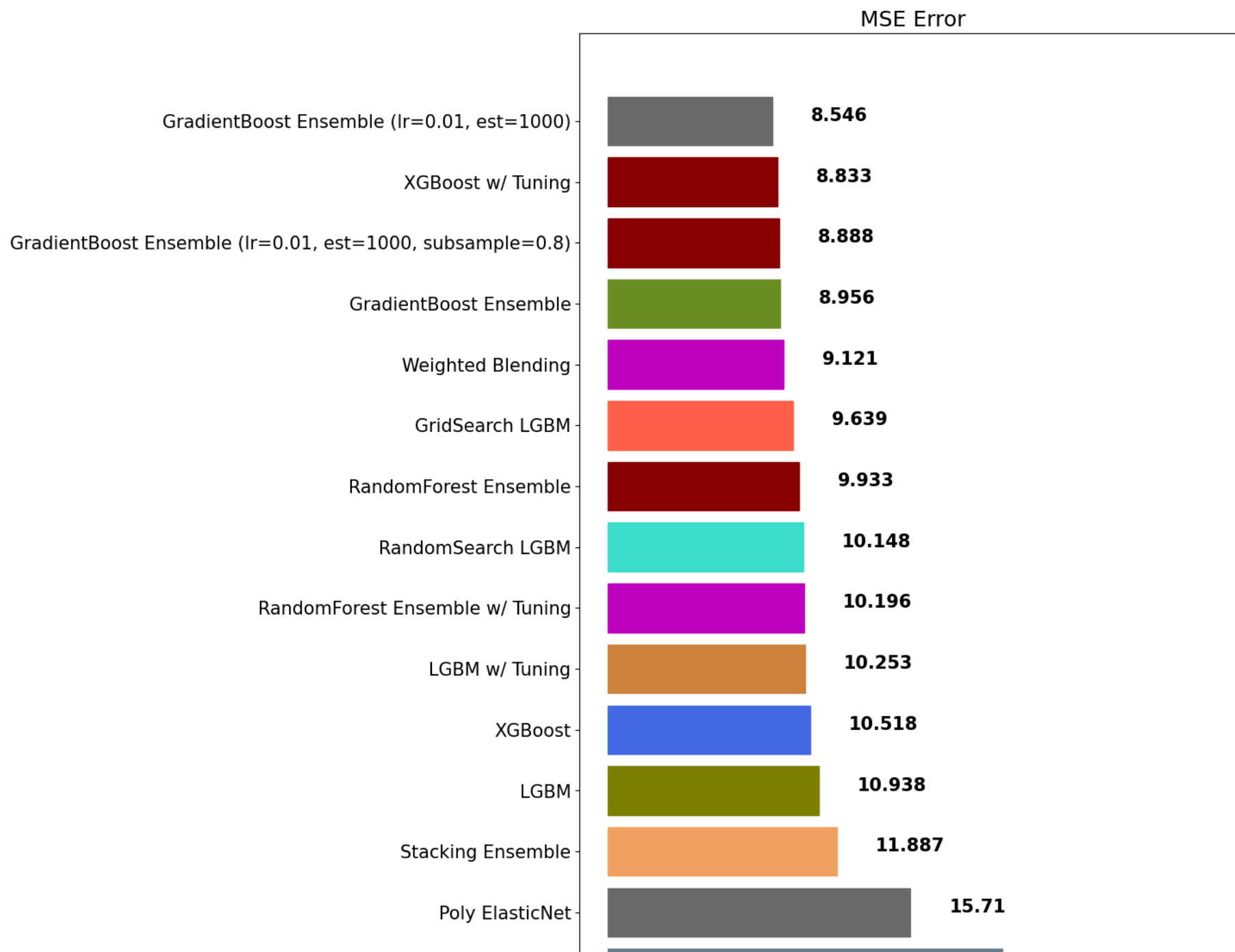
Out[160... {'colsample_bytree': 0.8,
 'learning_rate': 0.05,
 'max_depth': 7,
 'n_estimators': 500,
 'subsample': 0.8}

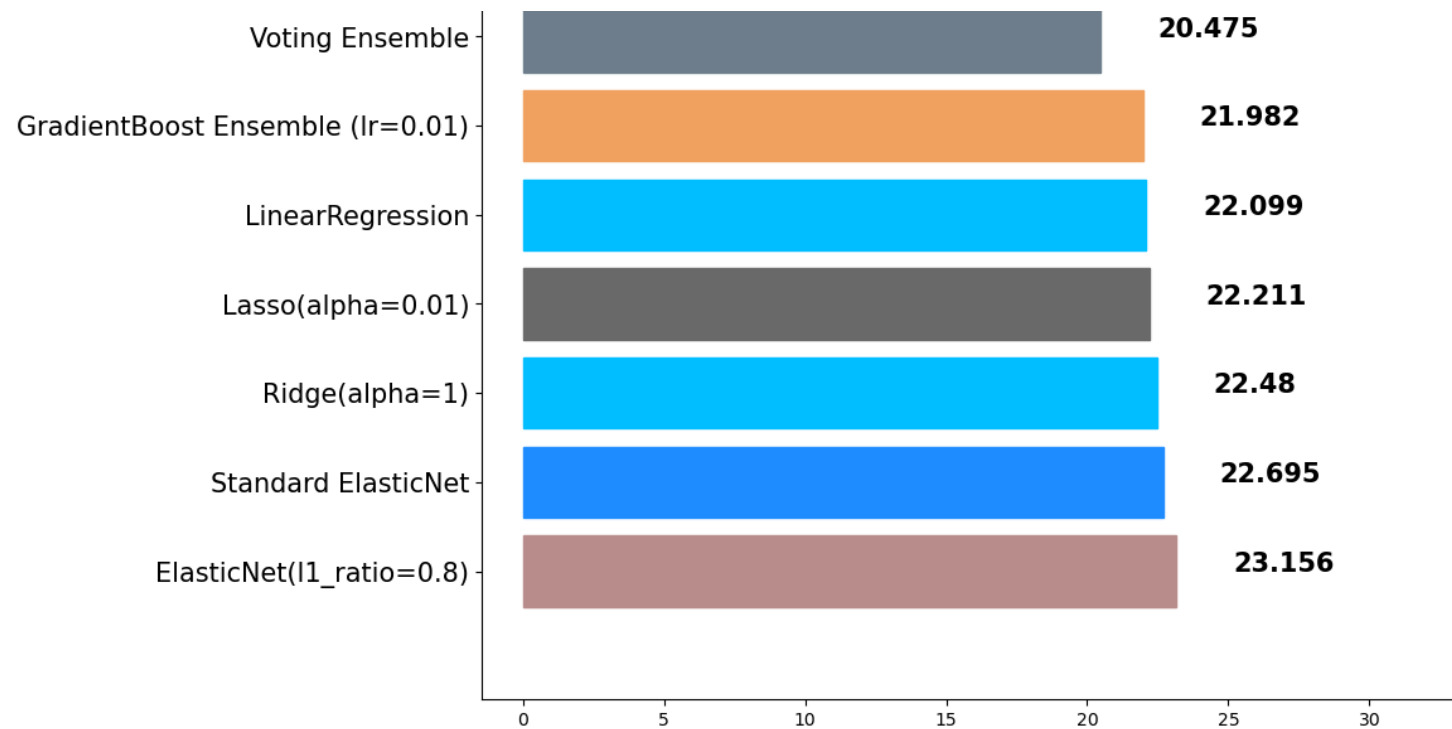
In [161... `lgbm_best = LGBMRegressor(n_estimators=500, subsample=0.8, max_depth=7, learning_rate=0.05, colsample_bytree=0.8)`
`lgbm_best_pred = lgbm_best.fit(x_train, y_train).predict(x_test)`
`mse_eval('GridSearch LGBM', lgbm_best_pred, y_test)`

GridSearch LGBM



	0	20	40	60	80	100	120
			model	mse			
0			ElasticNet(l1_ratio=0.8)	23.156236			
1			Standard ElasticNet	22.694970			
2			Ridge(alpha=1)	22.480476			
3			Lasso(alpha=0.01)	22.210557			
4			LinearRegression	22.098695			
5			GradientBoost Ensemble (lr=0.01)	21.982151			
6			Voting Ensemble	20.474757			
7			Poly ElasticNet	15.710492			
8			Stacking Ensemble	11.886880			
9			LGBM	10.937726			
10			XGBoost	10.518450			
11			LGBM w/ Tuning	10.252945			
12			RandomForest Ensemble w/ Tuning	10.196040			
13			RandomSearch LGBM	10.147690			
14			RandomForest Ensemble	9.933307			
15			GridSearch LGBM	9.639342			
16			Weighted Blending	9.121321			
17			GradientBoost Ensemble	8.956141			
18			GradientBoost Ensemble (lr=0.01, est=1000, sub...	8.887780			
19			XGBoost w/ Tuning	8.832910			
20			GradientBoost Ensemble (lr=0.01, est=1000)	8.546408			





In []:

In []:

In []: